

An Introduction to Recurrent Neural Networks

(and beyond!)

By: Ehsan Moradi

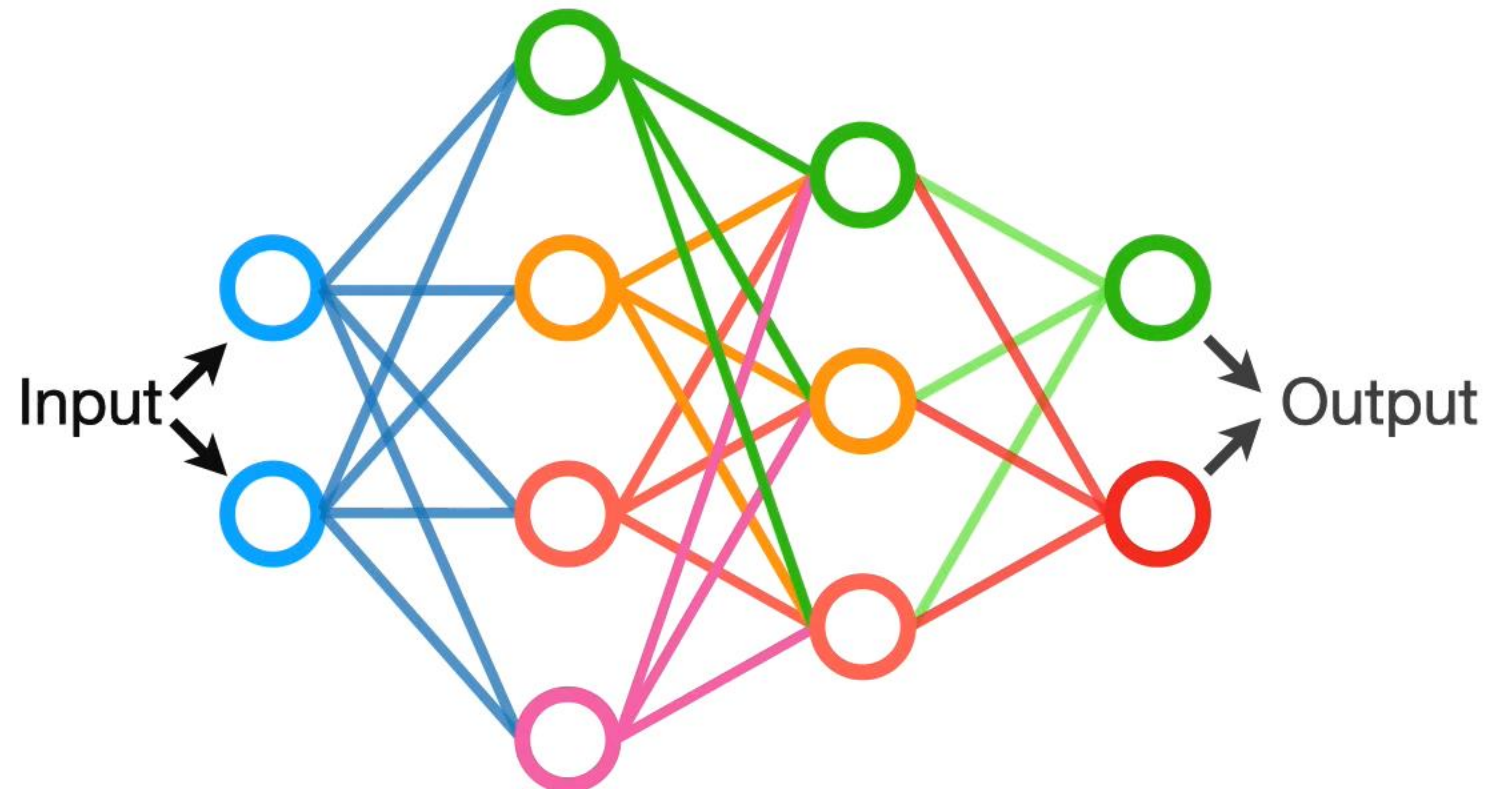
Amirkabir University of Technology (Tehran Polytechnic)

Applied Machine Learning - Spring 2025

Table of contents

- Neural Networks Revision
 - Problem Definition
 - The NN Architecture and Solution
- Recurrent Neural Networks
 - Use Cases
 - Another Sample Problem
 - How RNN Works
- Long Short-Term Memory (LSTM)
 - Deep Dive into How it Works

Neural Networks, one of the most popular algorithms in **Machine Learning**, cover a broad range of concepts and techniques.



So, with that said, let's imagine
we tested a drug that was
designed to treat an illness...



Vs.



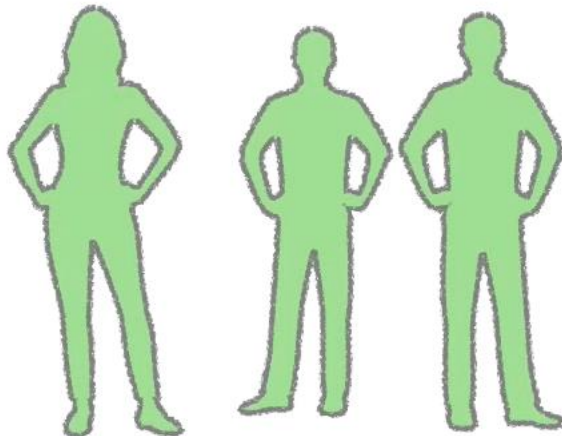
...and we gave the drug to 3
different groups of people with 3
different **Dosages**.



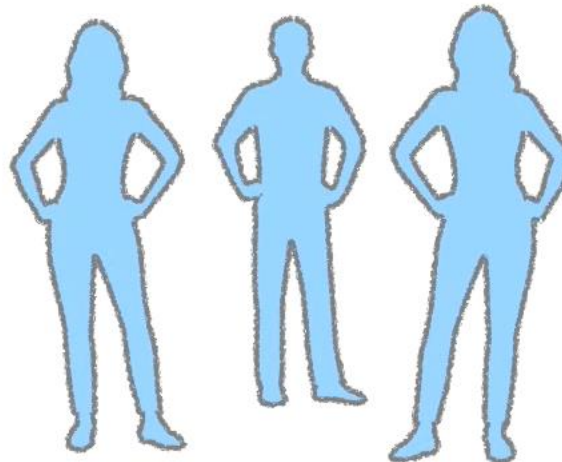
Vs.



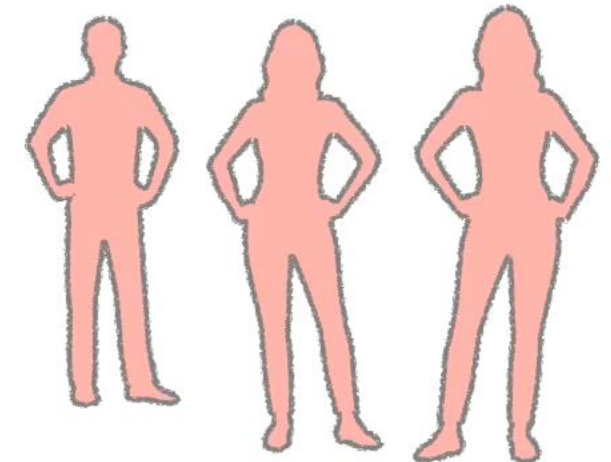
Low Dosage

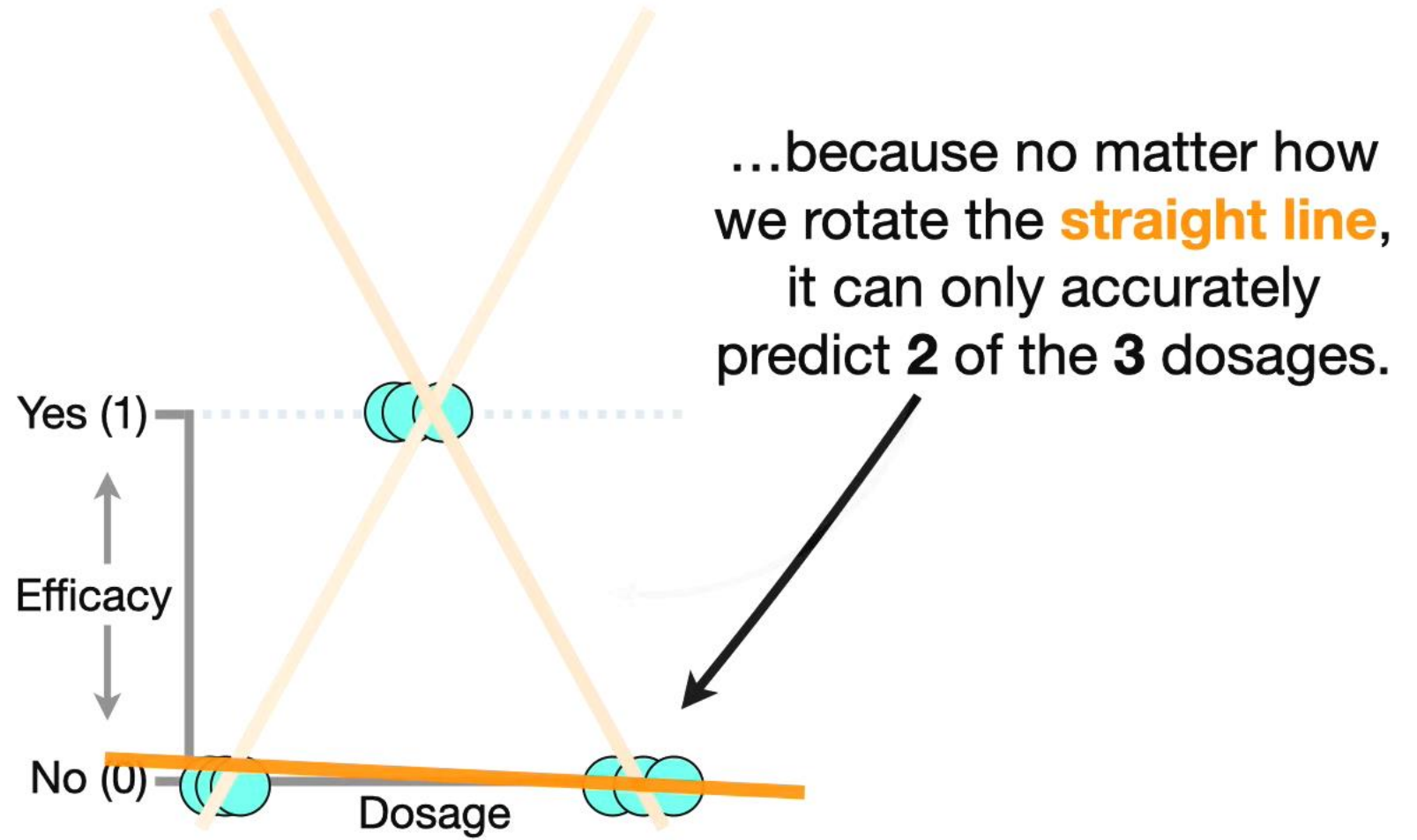


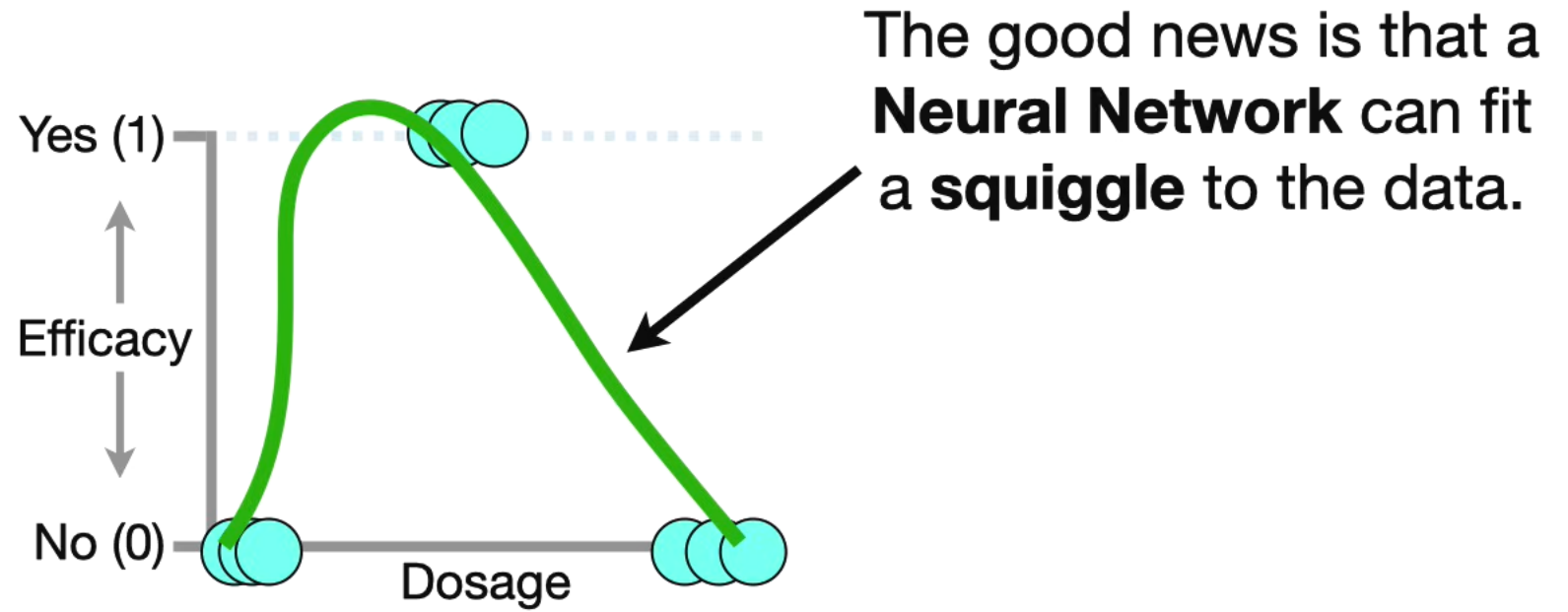
Medium Dosage



High Dosage

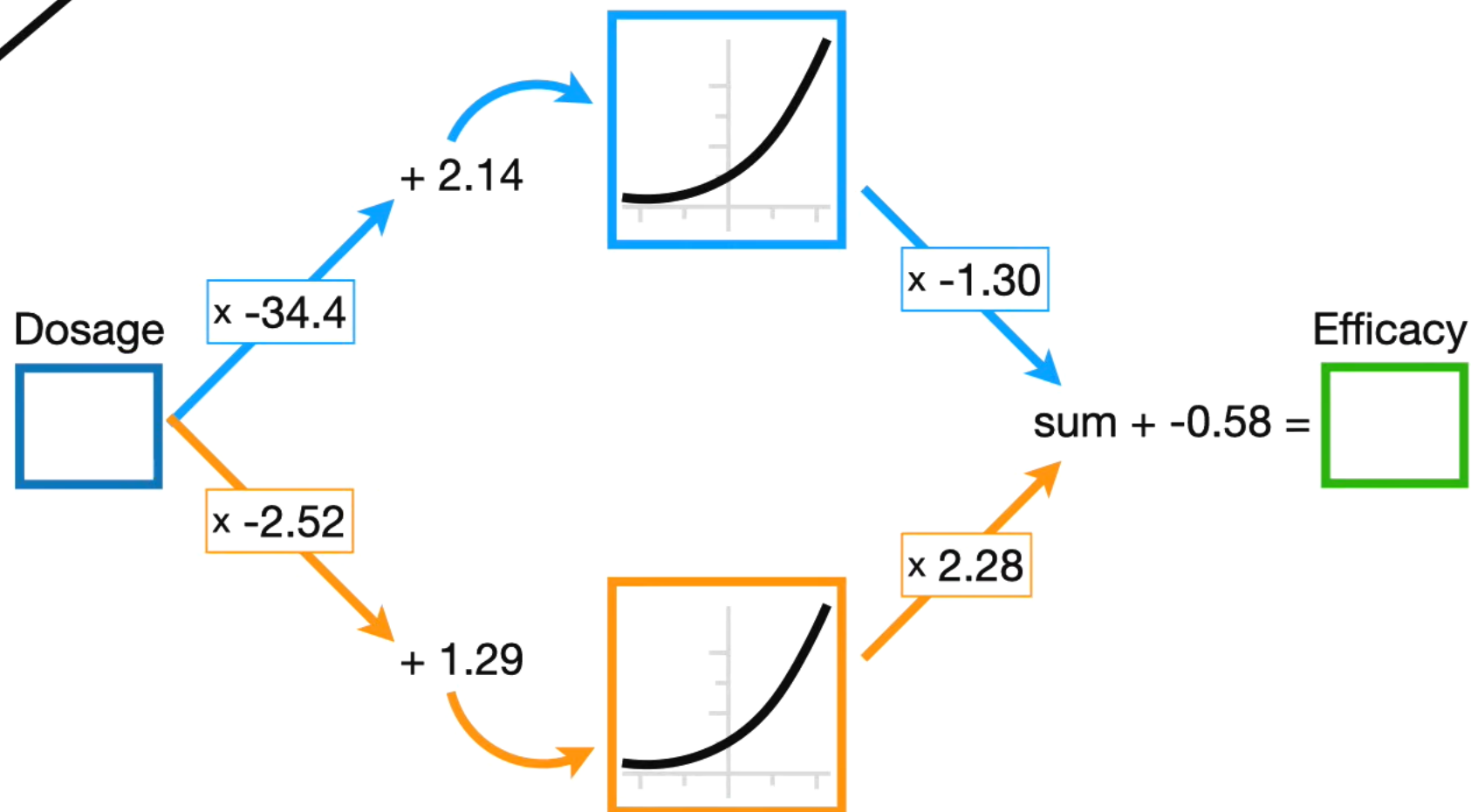
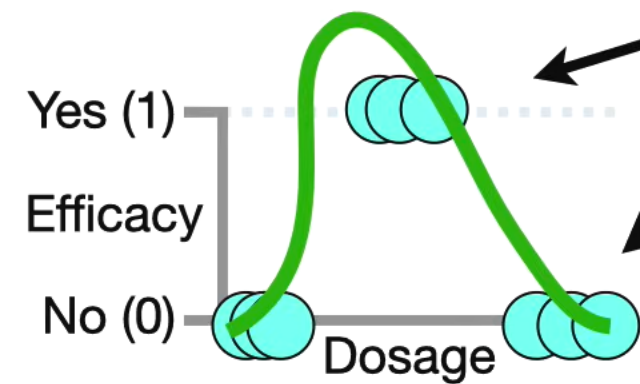


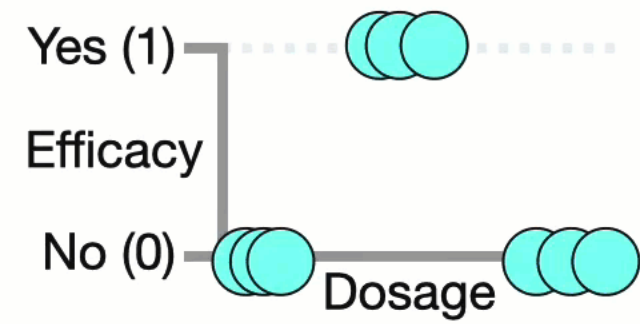




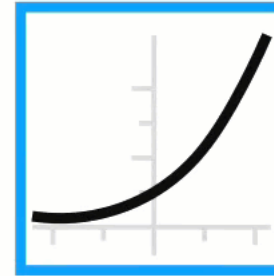


...creates this **green squiggle**.





+ 2.14



x -1.30

Dosage x -34.4



x -2.52

+ 1.29



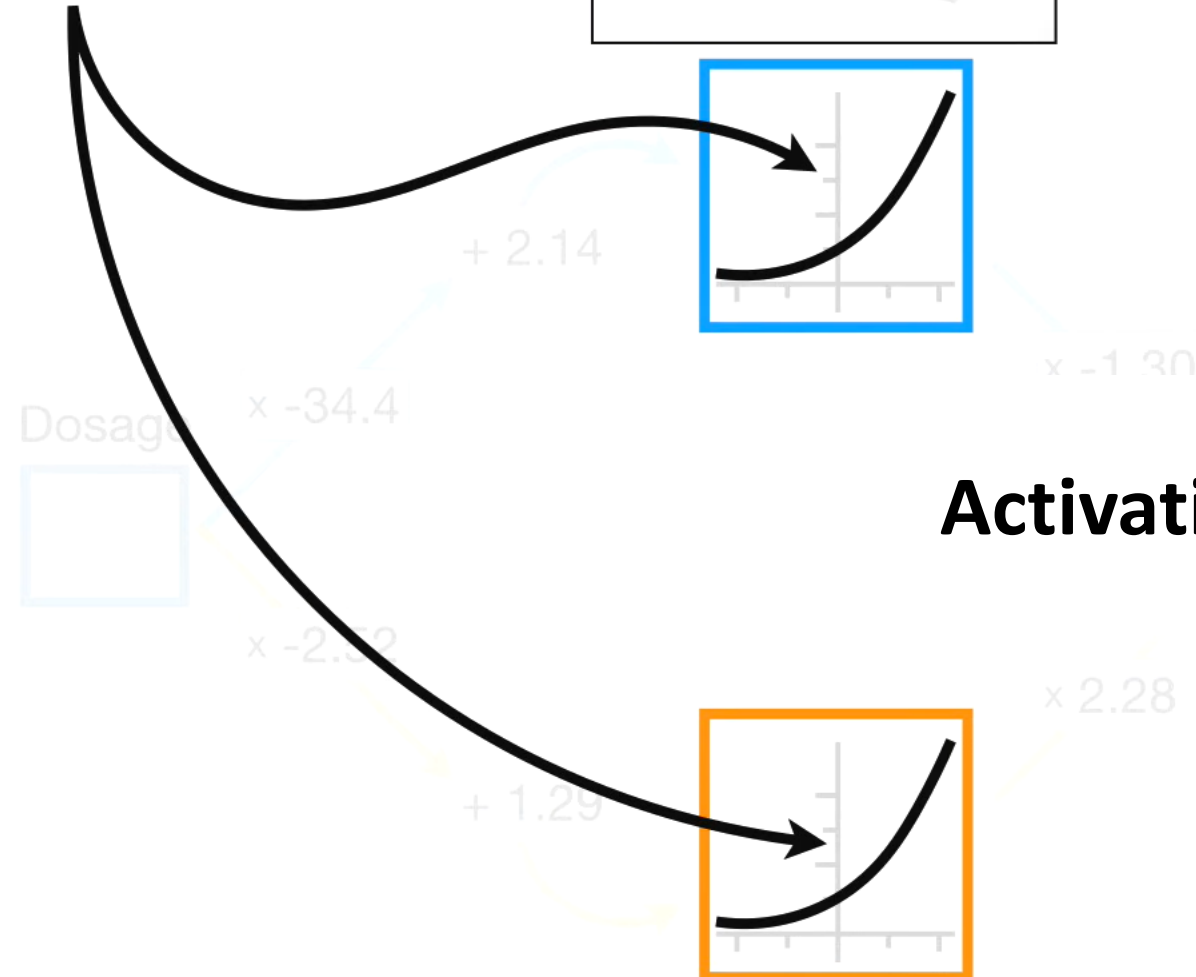
x 2.28

Efficacy

sum + -0.58 =



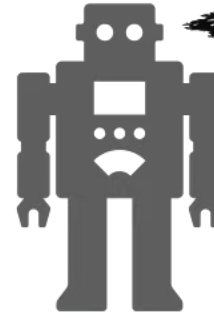
This specific **curved line** is called **softplus**, which sounds like a brand of toilet paper.



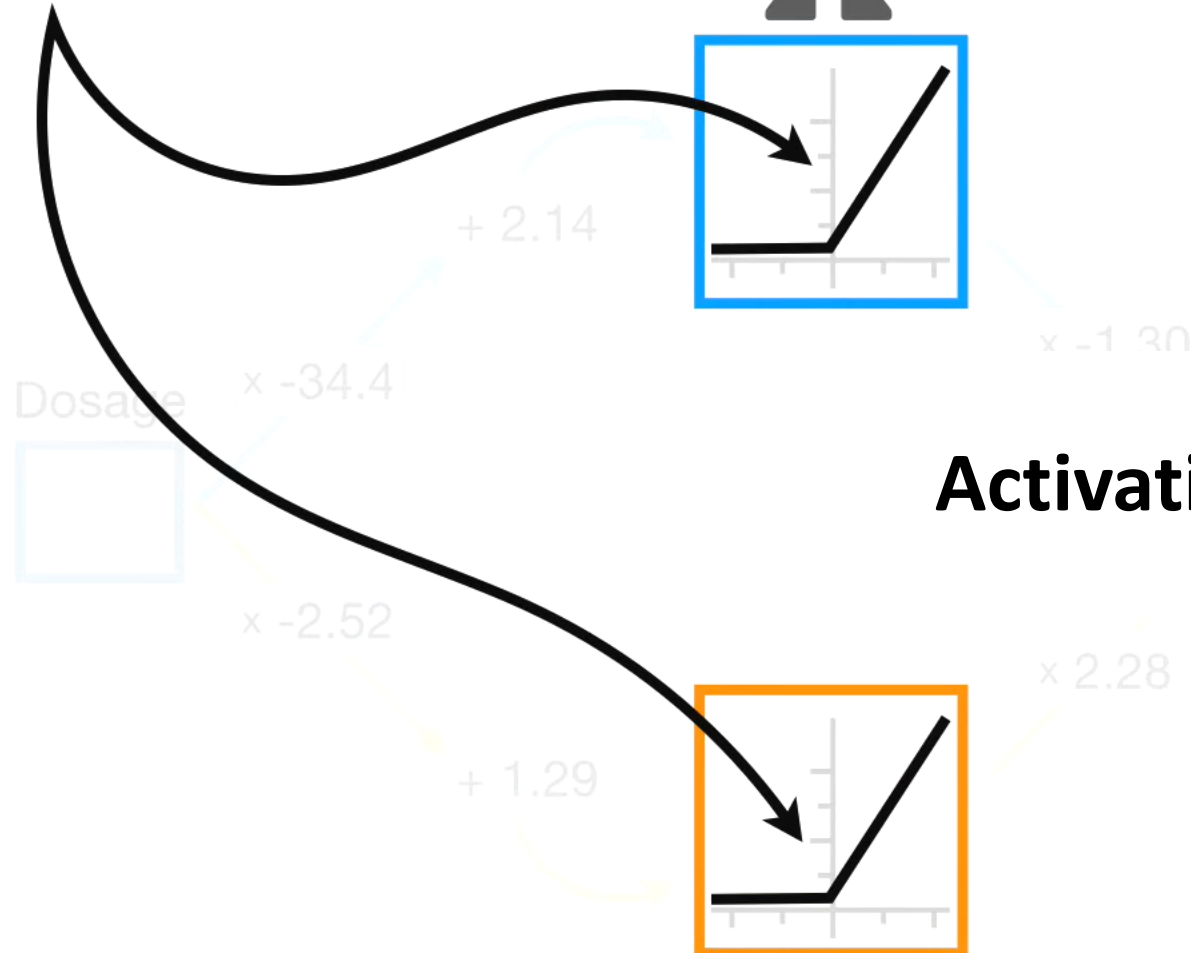
Activation Function!

Alternatively, we could use this **bent line**, called, **ReLU**, which is short for **Rectified Linear Unit** and sounds like a robot.

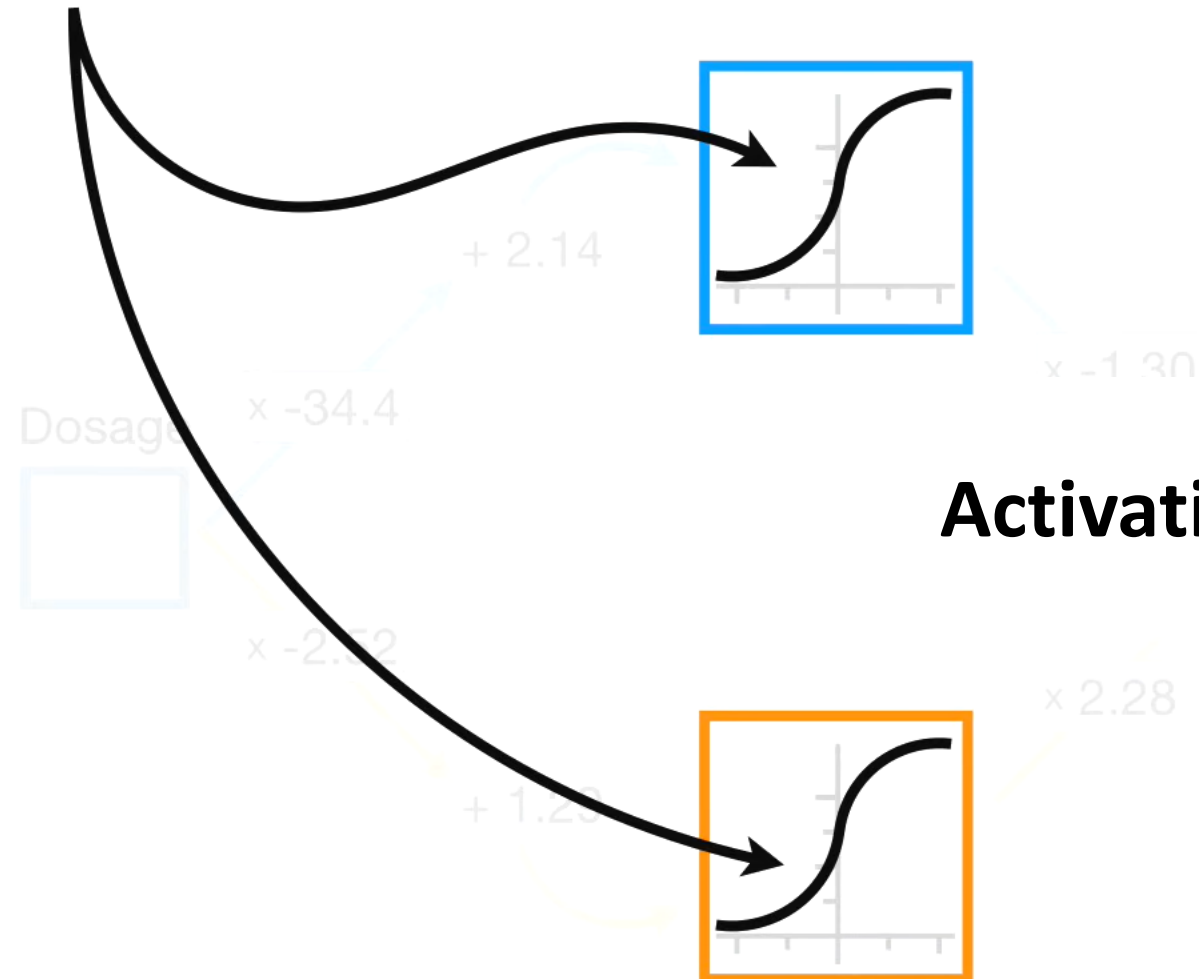
ReLU



Take me to
your leader!



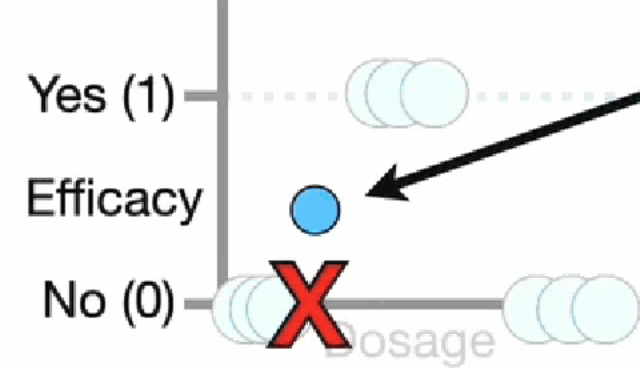
Or we could use a **sigmoid** shape or any other **bent** or **curved line**.



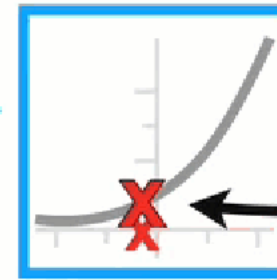
Activation Function!

$$(0.1 \times -34.4) + 2.14 = -1.3$$

So let's put a **blue dot** at **0.24** for when **Dosage = 0.1**.



$$f(-1.3) = \log(1 + e^{-1.3}) = 0.24$$



Dosage

0.1

$$\times -34.4$$

$$+ 2.14$$

$$\times -1.30$$

Efficacy

$$\text{sum} + -0.58 =$$

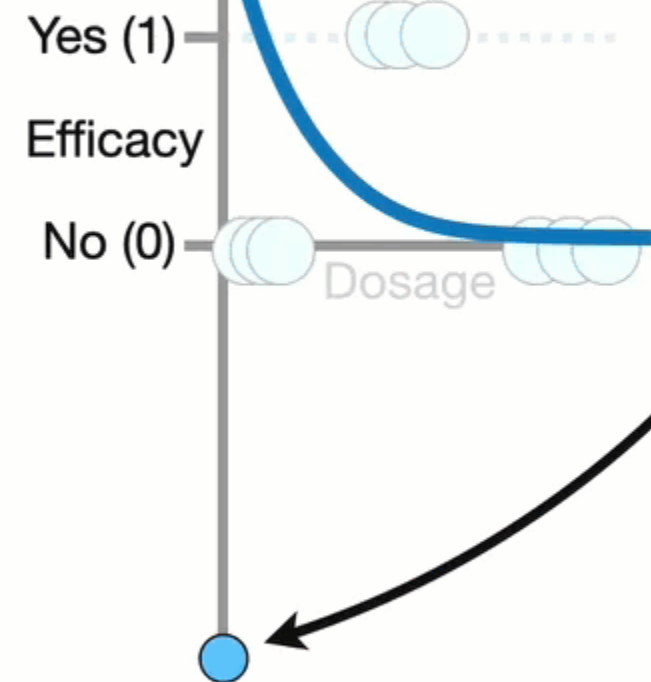
$$\times 2.28$$

$$+ 1.29$$



...and **-2.93** corresponds to
this position on the y-axis.

$$2.25 \times -1.30 = -2.93$$

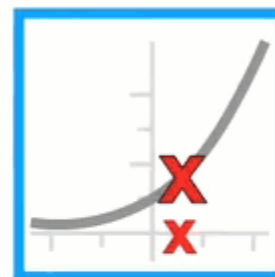


Dosage

0

$$\times -34.4$$

$$+ 2.14$$



$$\times -1.30$$

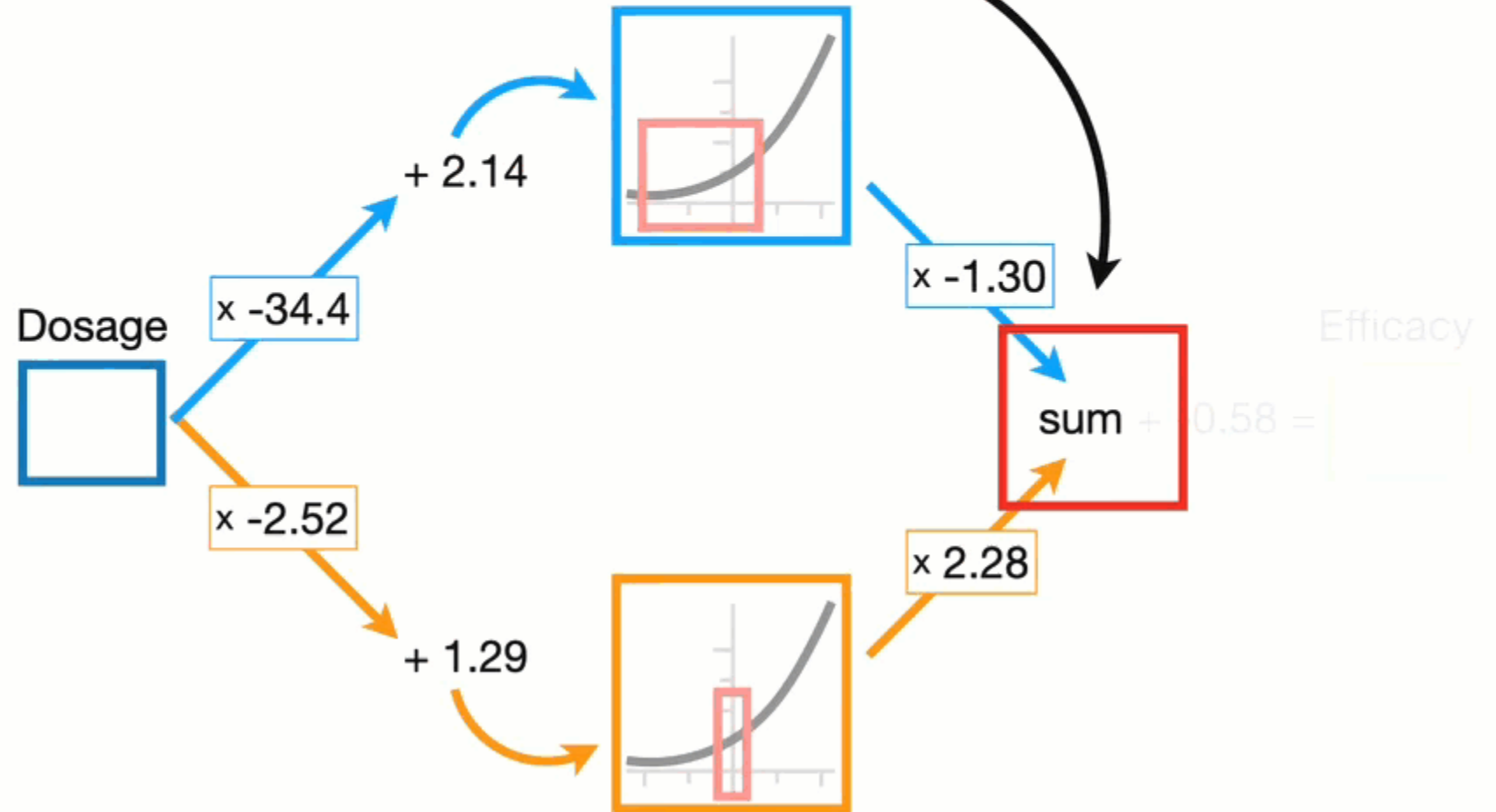
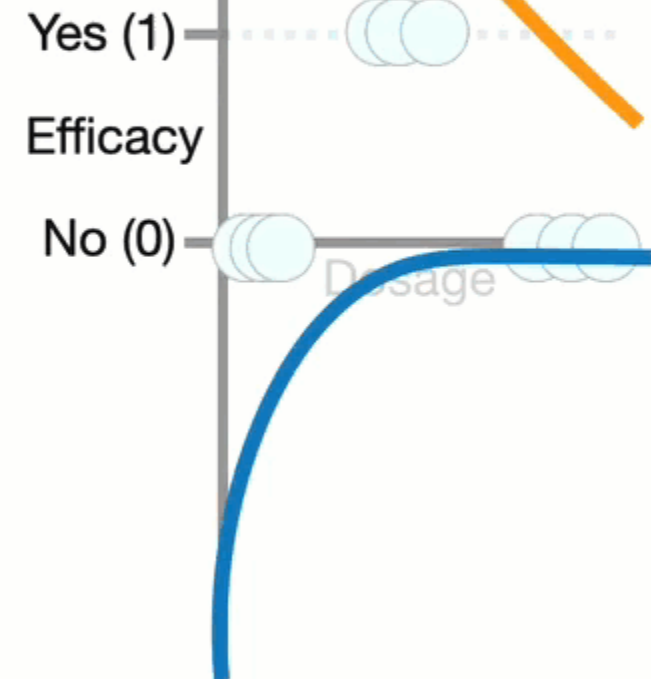
$$\text{sum} + -0.58 =$$

Efficacy

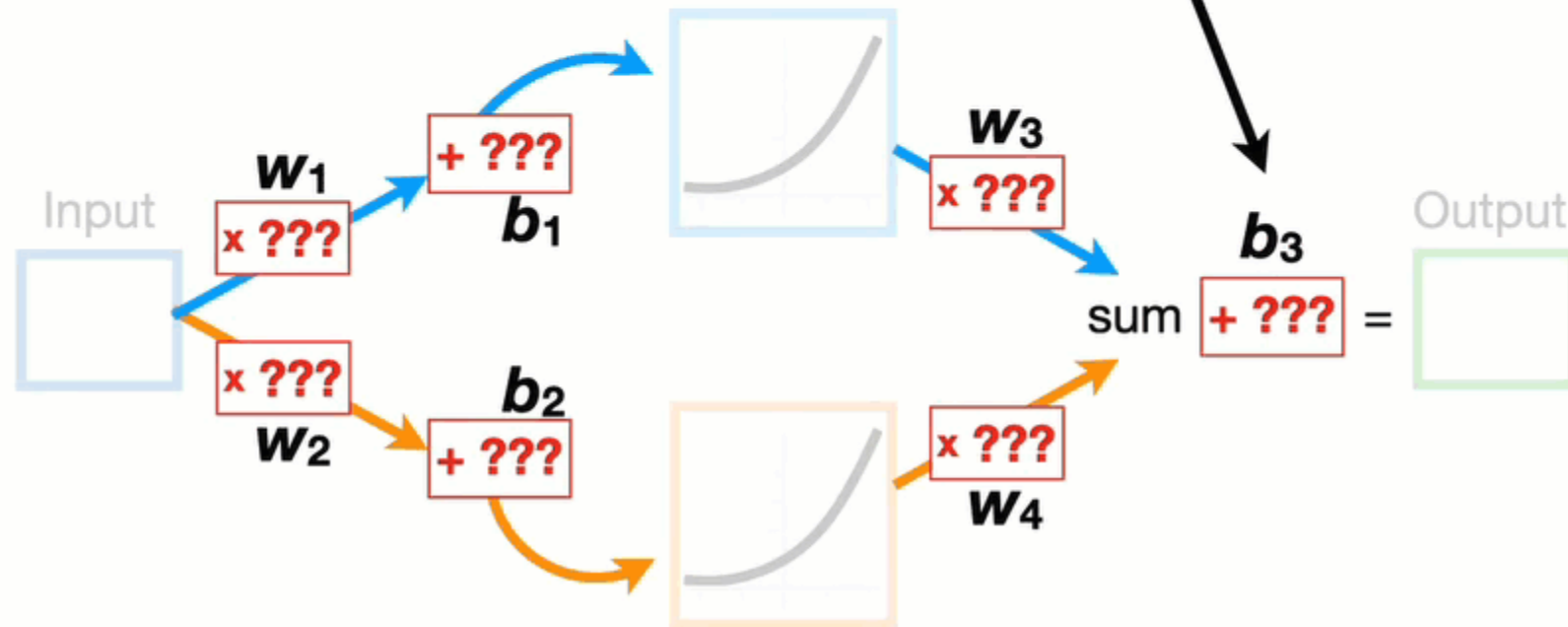
$$\times 2.28$$

$$+ 1.29$$

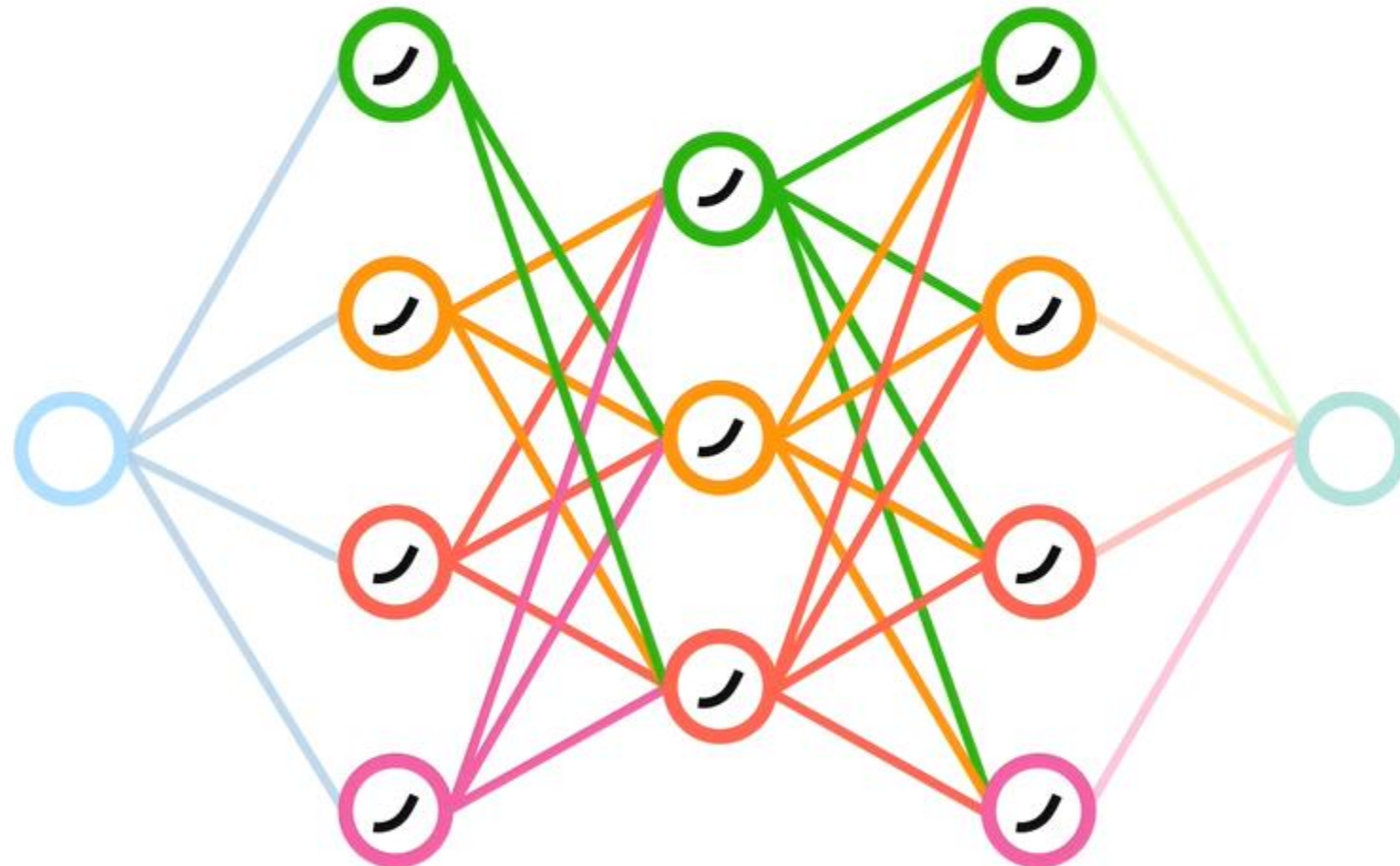
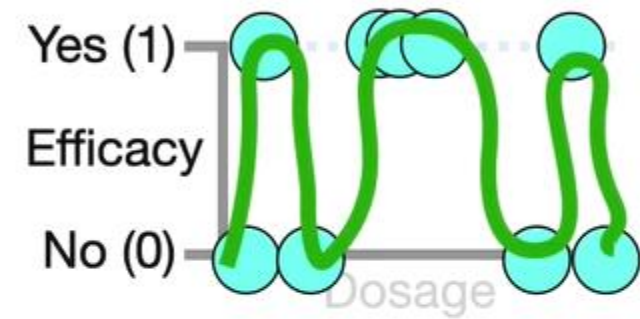
Now the **Neural Network** tells us to add the y-axis coordinates from **blue curve** to the **orange curve**...



Backpropagation



...just imagine what types of **green squiggles** we could fit with more **Hidden Layers** and more **Nodes** in each **Hidden Layer**.



Recurrent Neural Networks!

Examples of sequence data

Speech recognition



“The quick brown fox jumped
over the lazy dog.”

Music generation

∅



Sentiment classification

“There is nothing to like
in this movie.”



DNA sequence analysis

AGCCCCTGTGAGGAACTAG



AG**CCCCTGTGAGGAACT**AG

Machine translation

Voulez-vous chanter avec
moi?



Do you want to sing with
me?

Video activity recognition



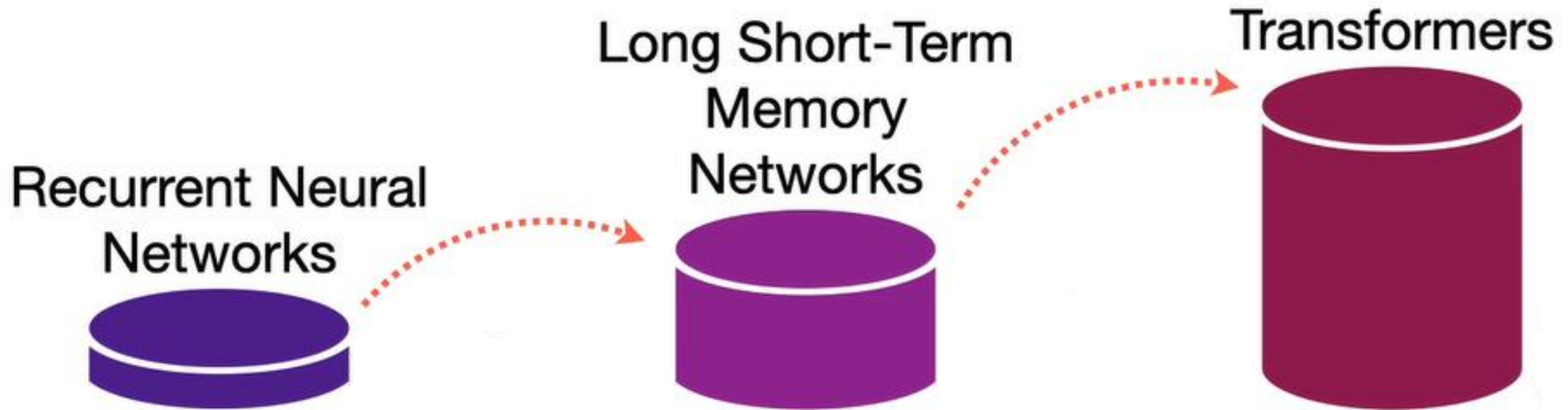
Running

Name entity recognition

Yesterday, Harry Potter
met Hermione Granger.

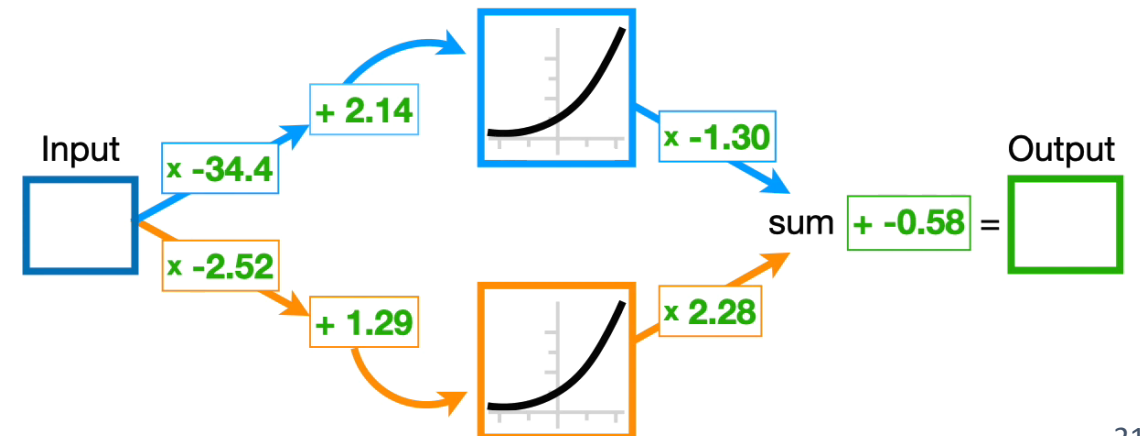
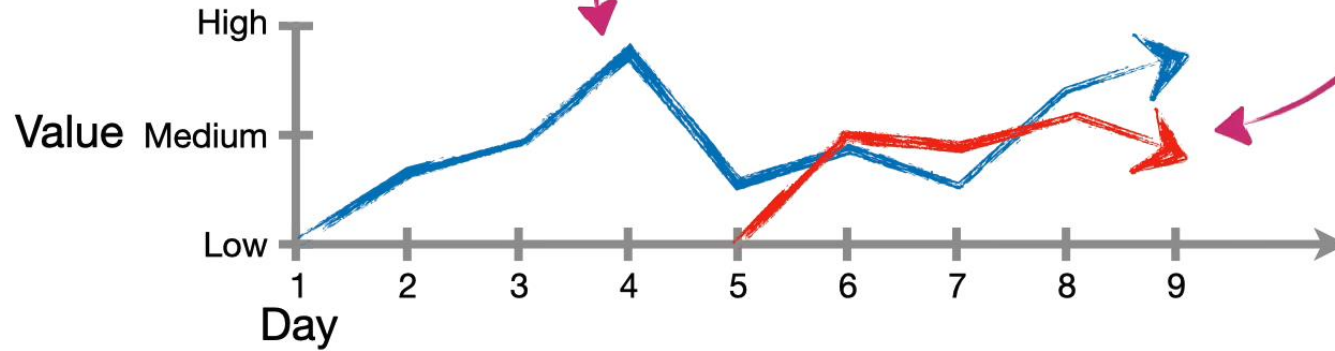


Yesterday, **Harry Potter**
met **Hermione Granger**.

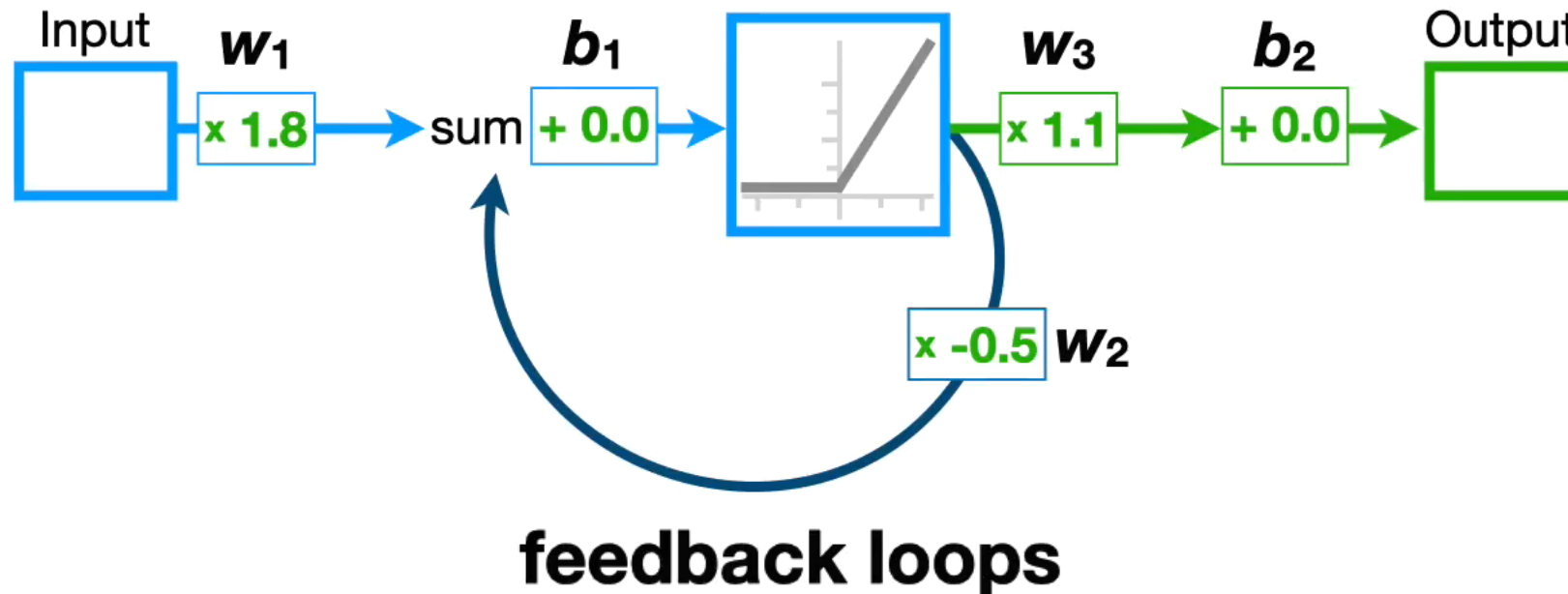


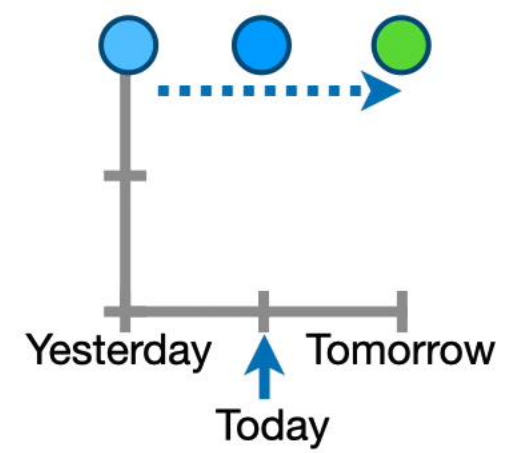
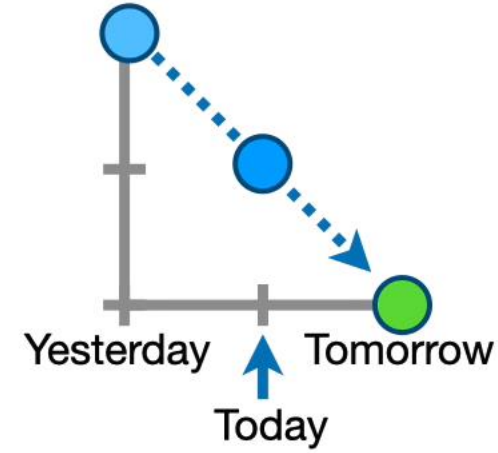
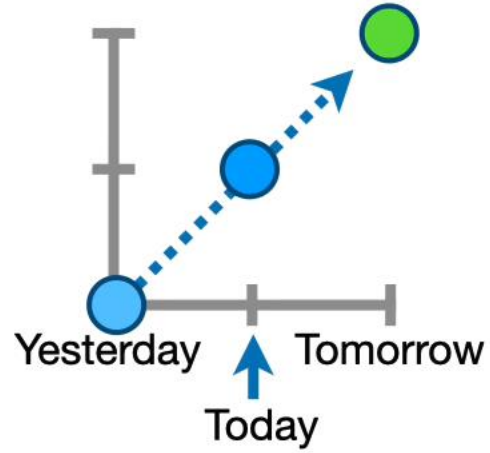
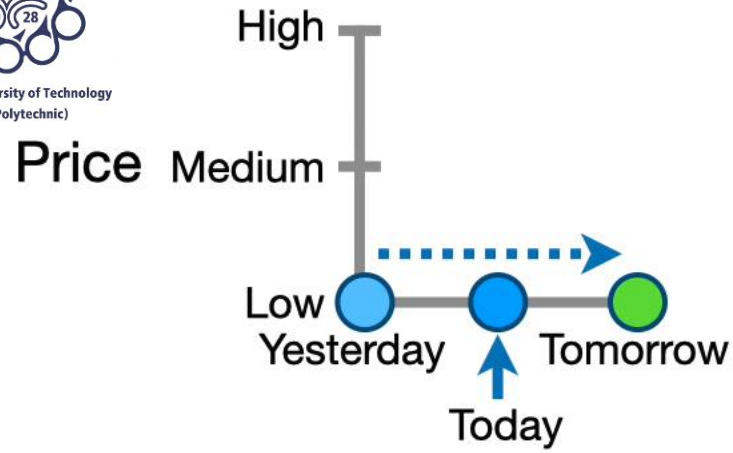
For example, we have more time points for the company represented by the **blue line**...

...then we have for the company represented by the **red line**.

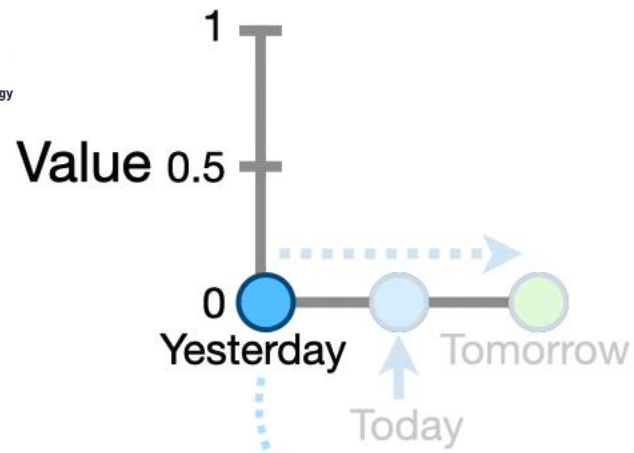


Just like the other neural networks that we've seen before, **Recurrent Neural Networks** have **weights**, **biases**,





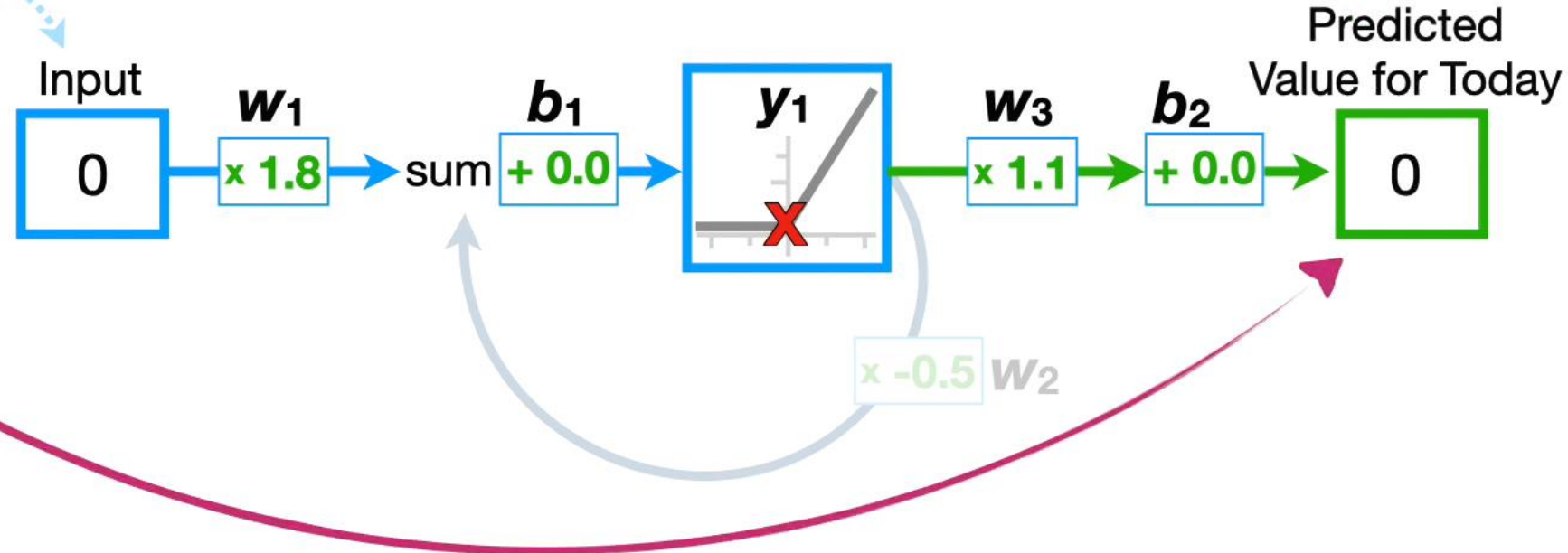
...we can talk about how to run
yesterday and **today's** data through
a **Recurrent Neural Network** to
predict **tomorrow's** price.

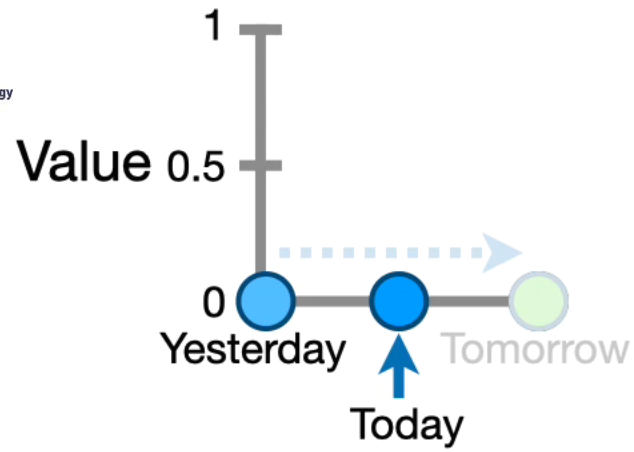


$$y_1 \times w_3 + b_2 = \text{The Predicted Value for Today}$$

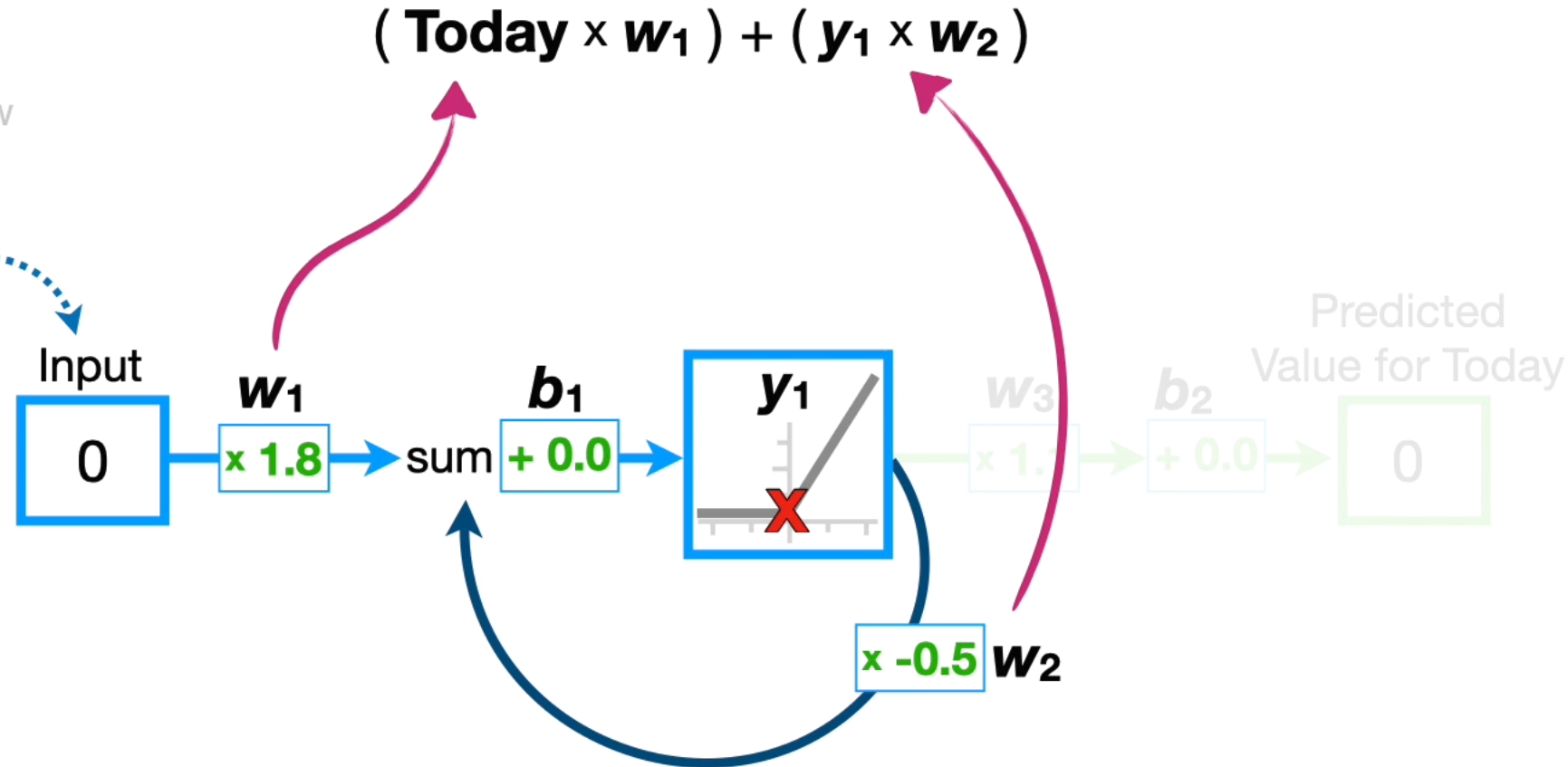
$$0 \times 1.1 + 0.0 = 0$$

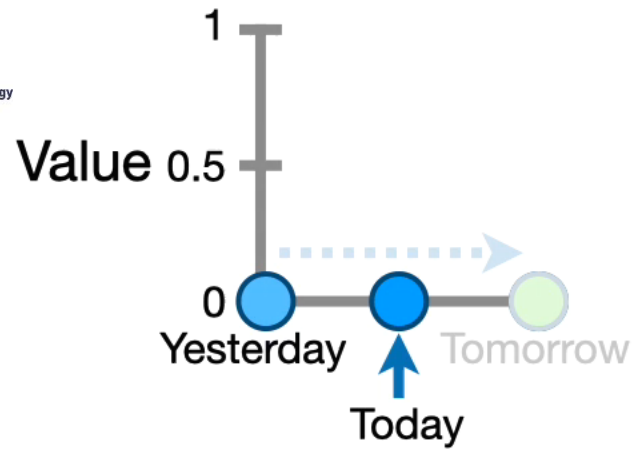
...then the output is the predicted value for **today**.



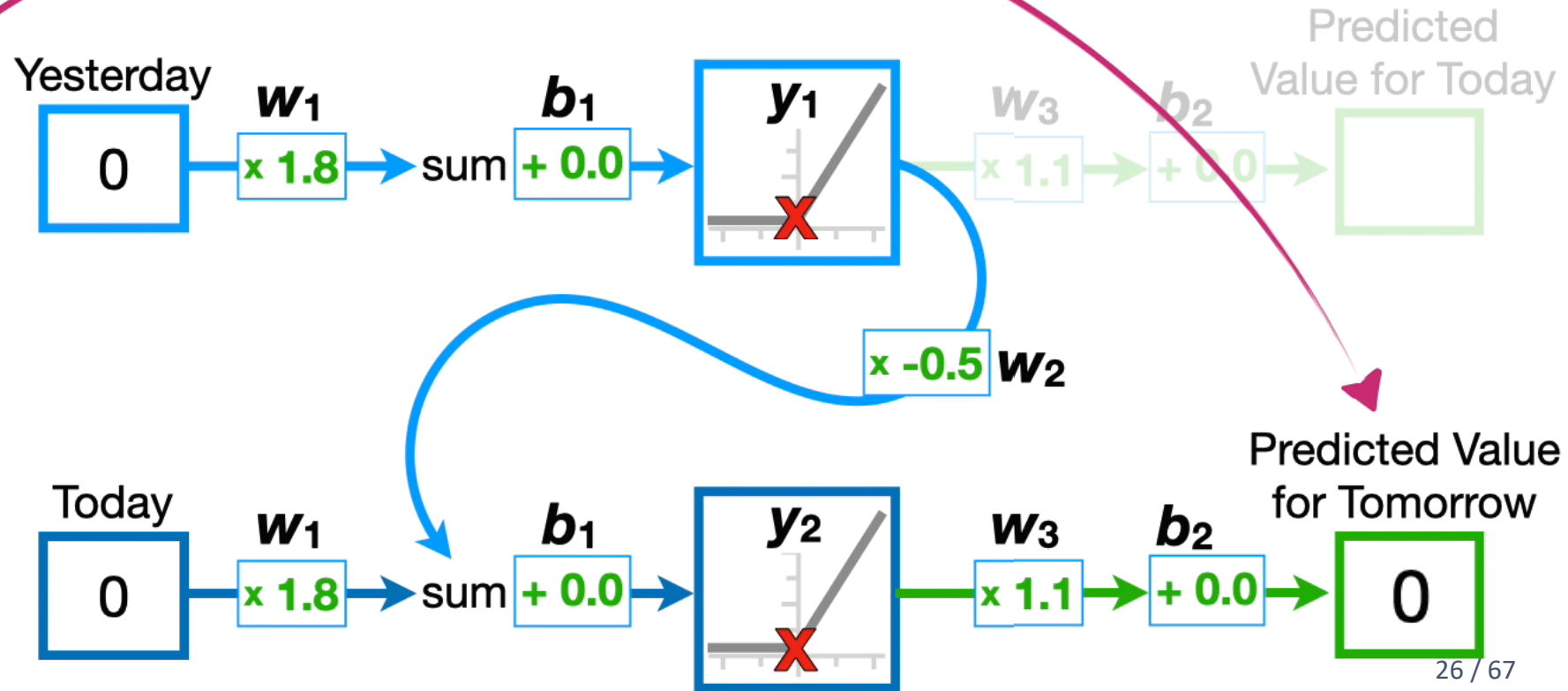


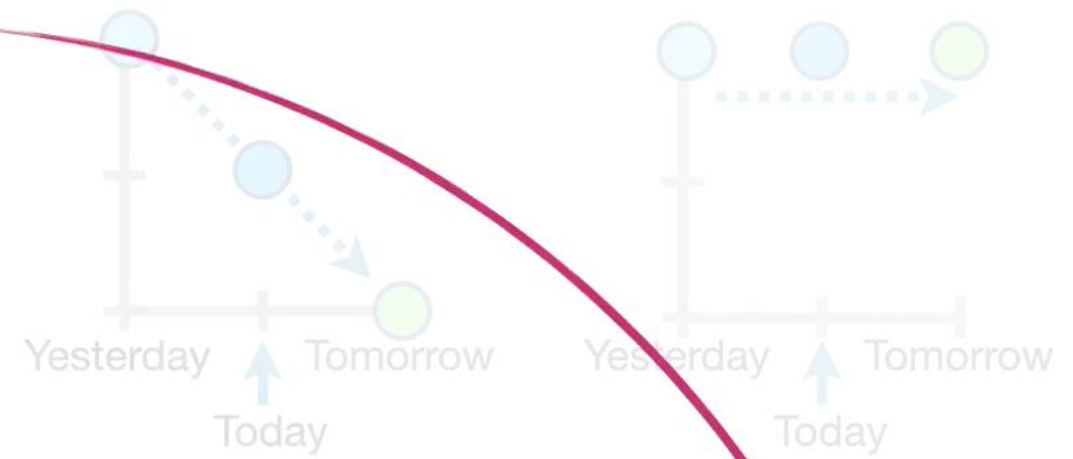
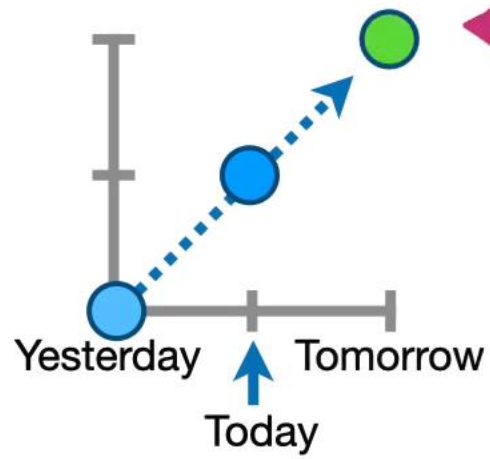
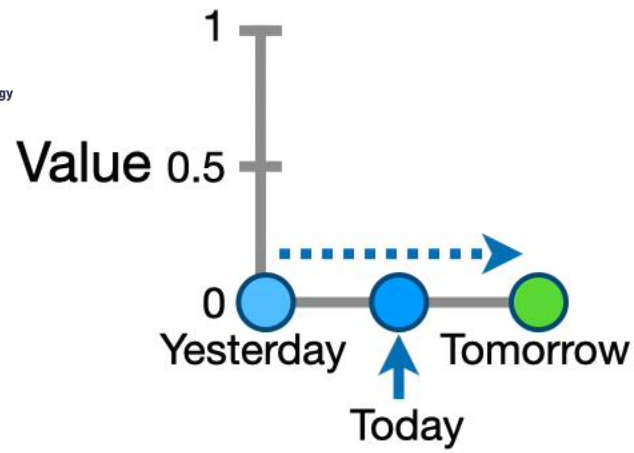
In other words, the **feedback loop** allows both **yesterday** and **today's** values to influence the prediction.



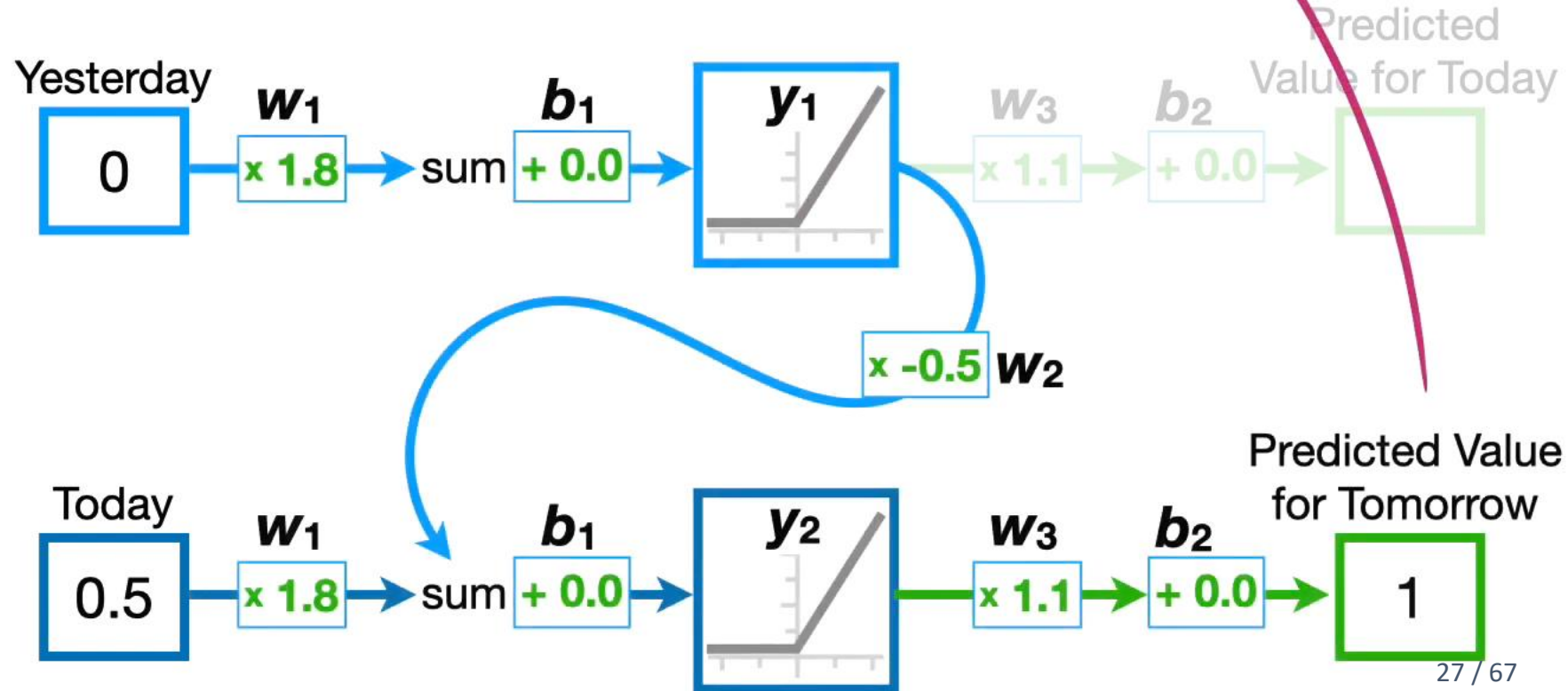


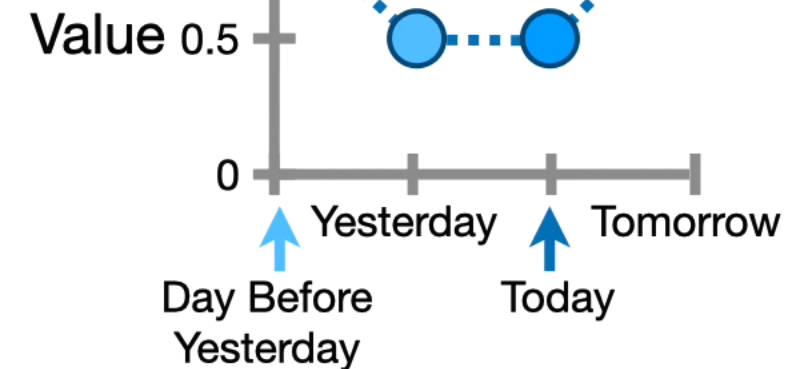
...which gives us the predicted value for tomorrow.



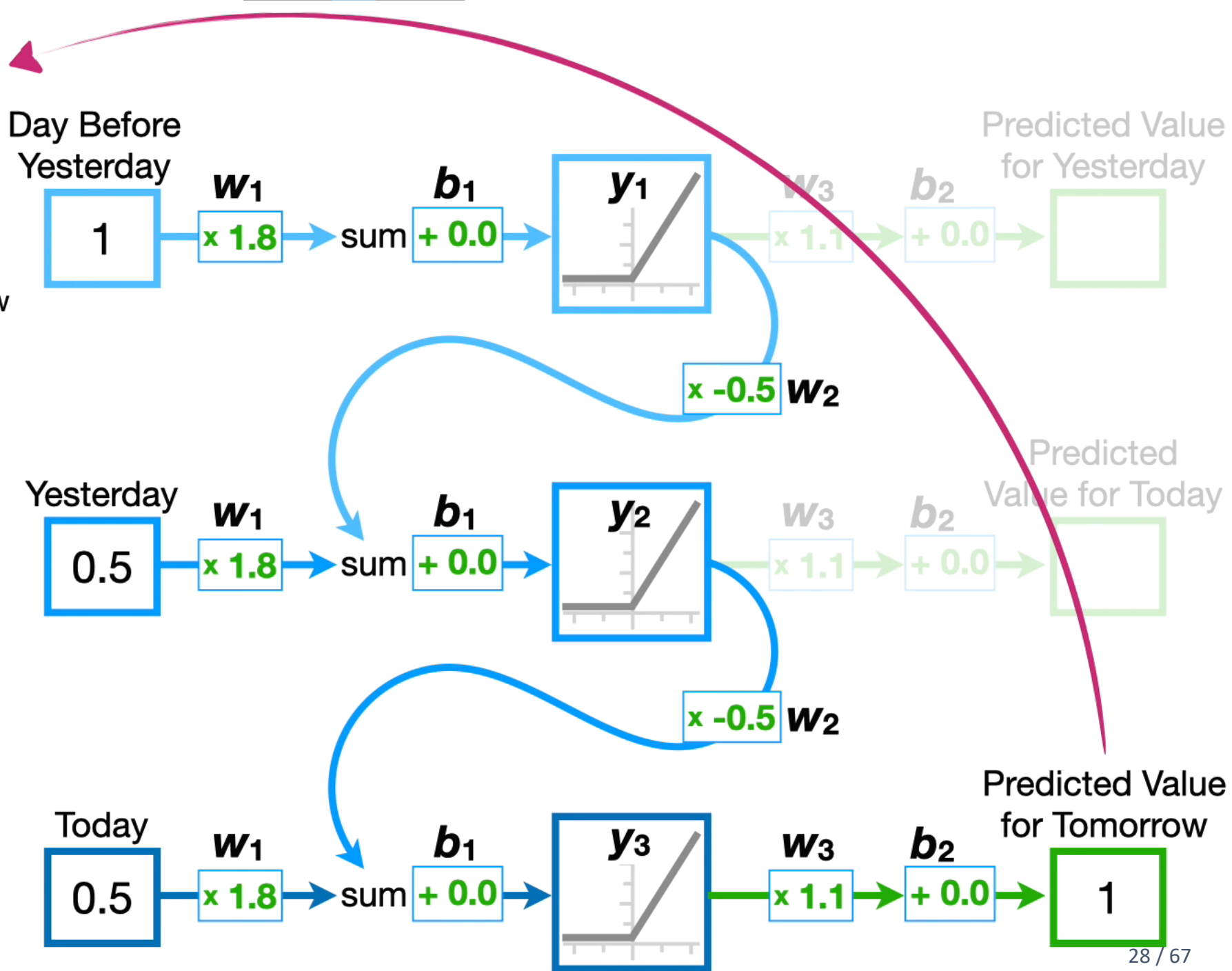


Likewise, when we run **yesterday** and **today's** values for the other scenarios through the recurrent neural network we predict the correct values for **tomorrow**.



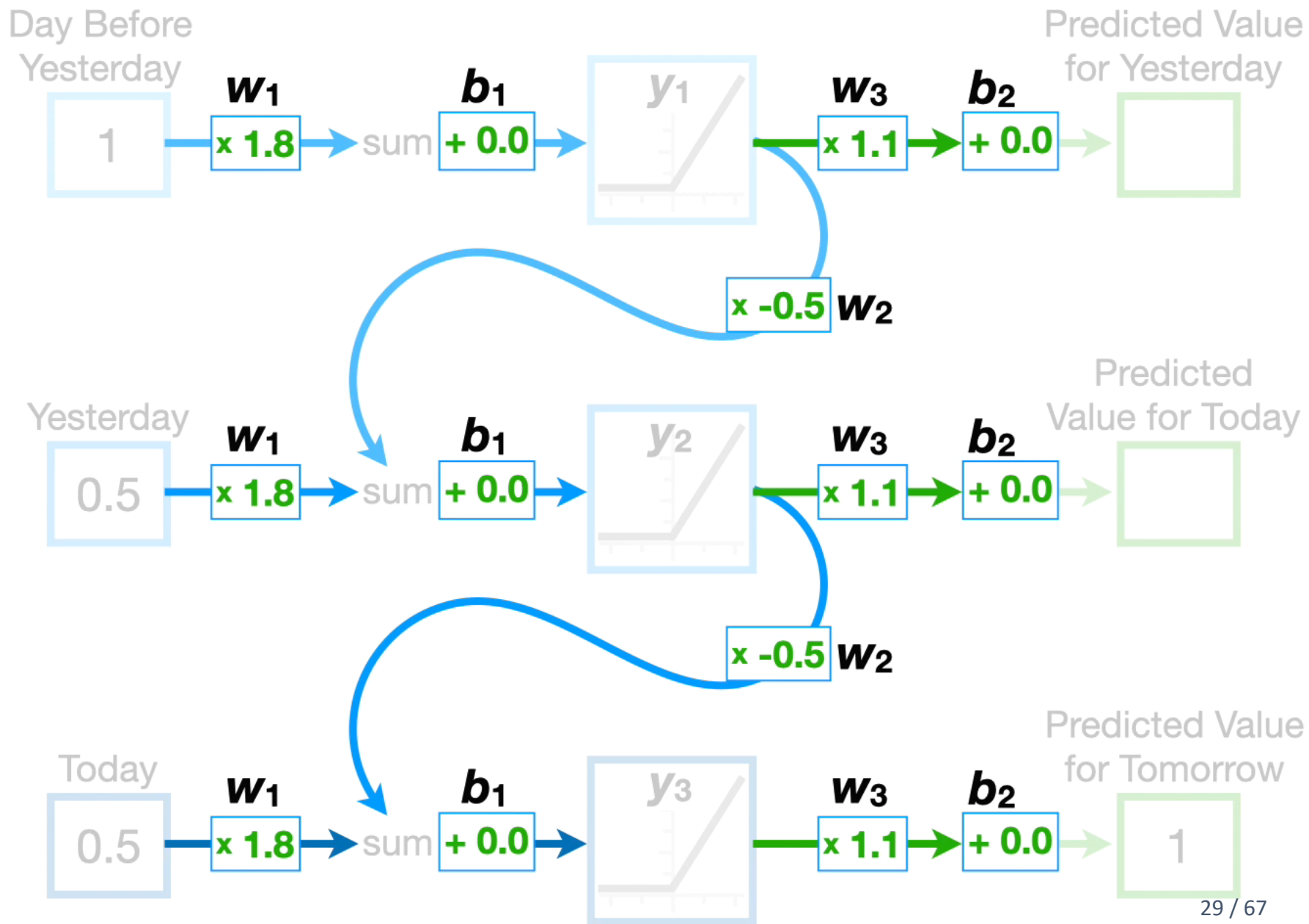


And when we do the math, the last output gives us the prediction for **tomorrow**.

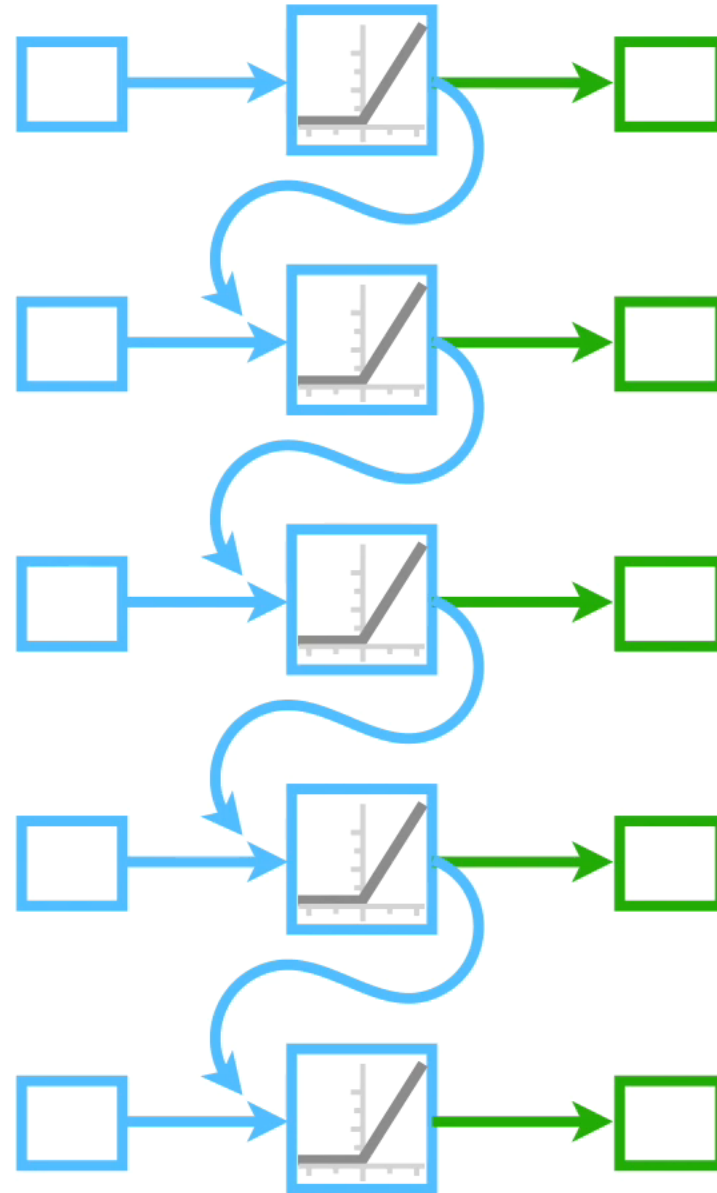




So, no matter how many times we **unroll** a recurrent neural network, we never increase the number of **weights** and **biases** that we have to train.



Vanishing/Exploding Gradient Problem.

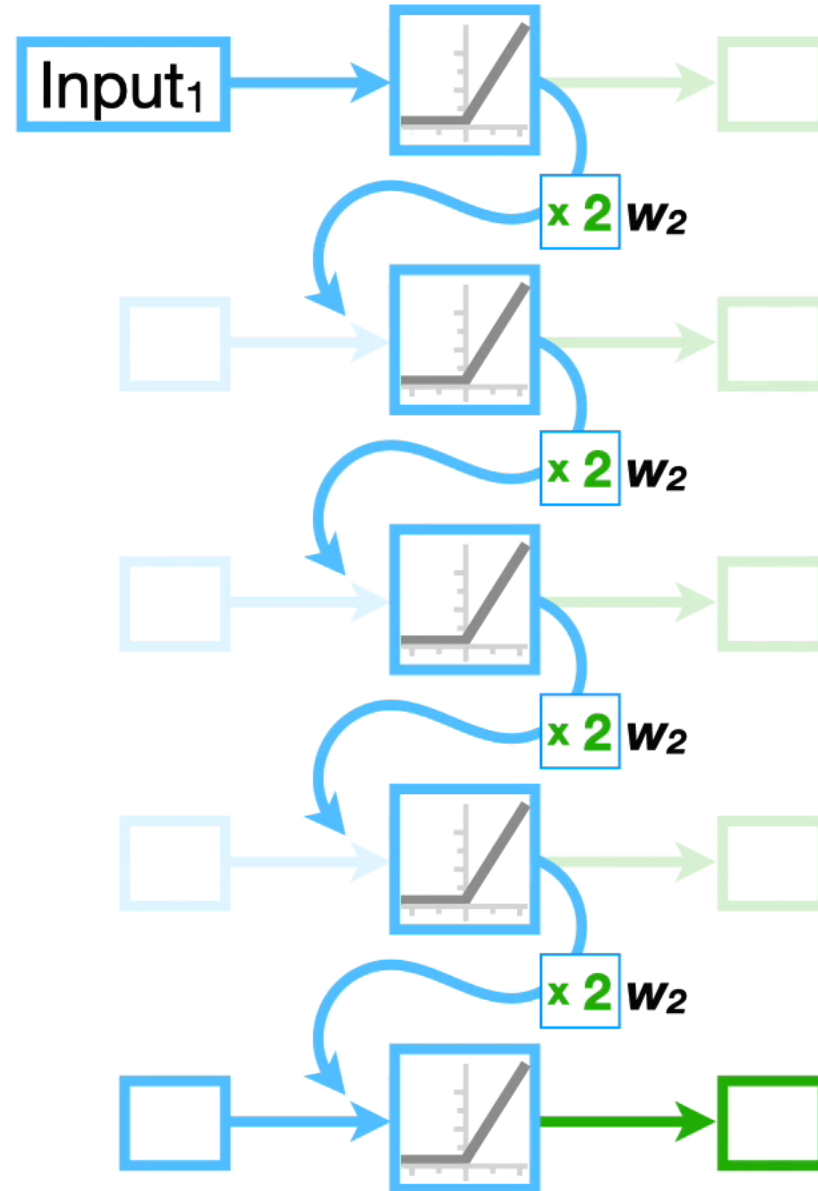


And that means the first input value is amplified **16** times before it gets to the final copy of the network.

$$\text{Input}_1 \times 2 \times 2 \times 2 \times 2$$

$$= \text{Input}_1 \times 2^4 = \text{Input}_1 \times 16$$

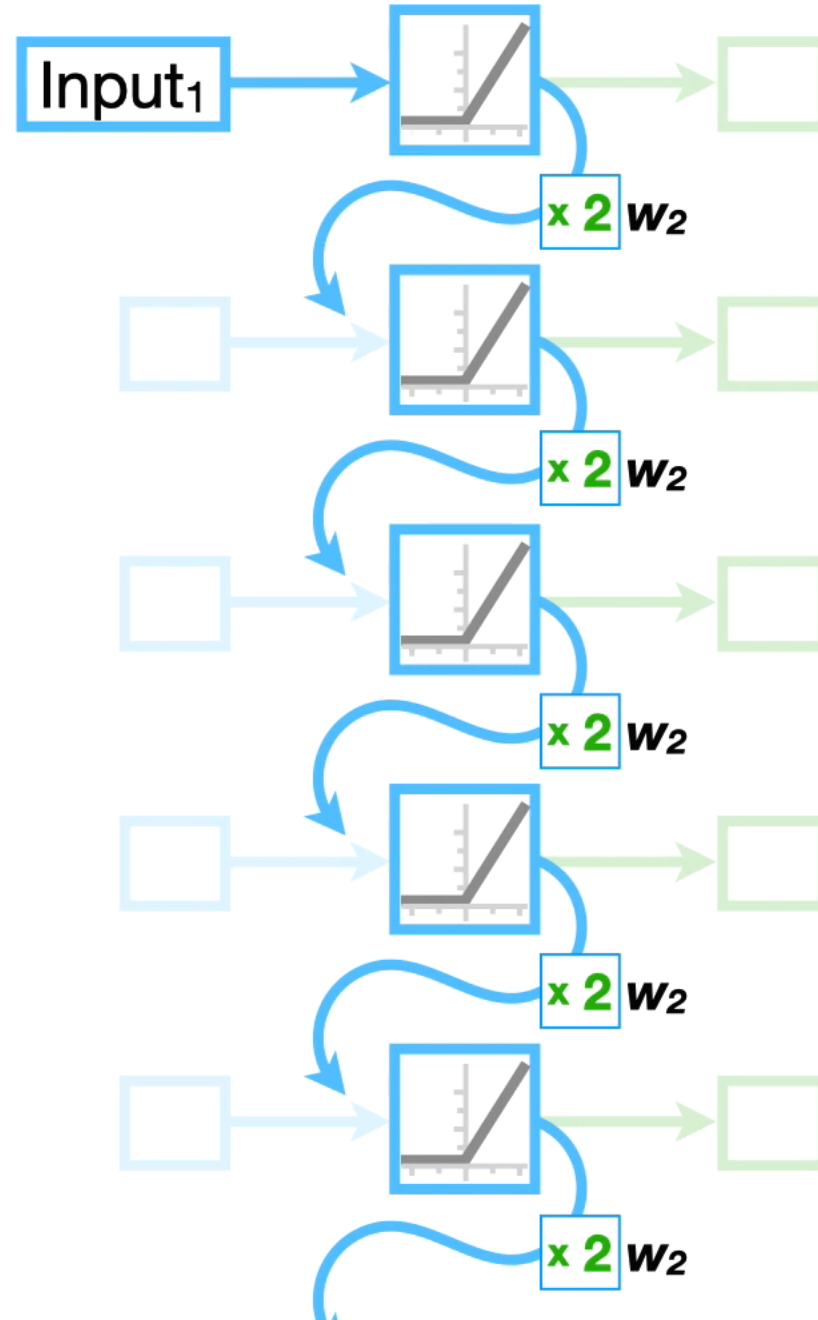
$$= \text{Input}_1 \times w_2^{\text{Num. Unroll}}$$



Now, if we had **50**
sequential days of stock
market data, which, to be
honest, really isn't that
much data...

$$= \text{Input}_1 \times 2^{50}$$

$$= \text{Input}_1 \times w_2^{\text{Num. Unroll}}$$

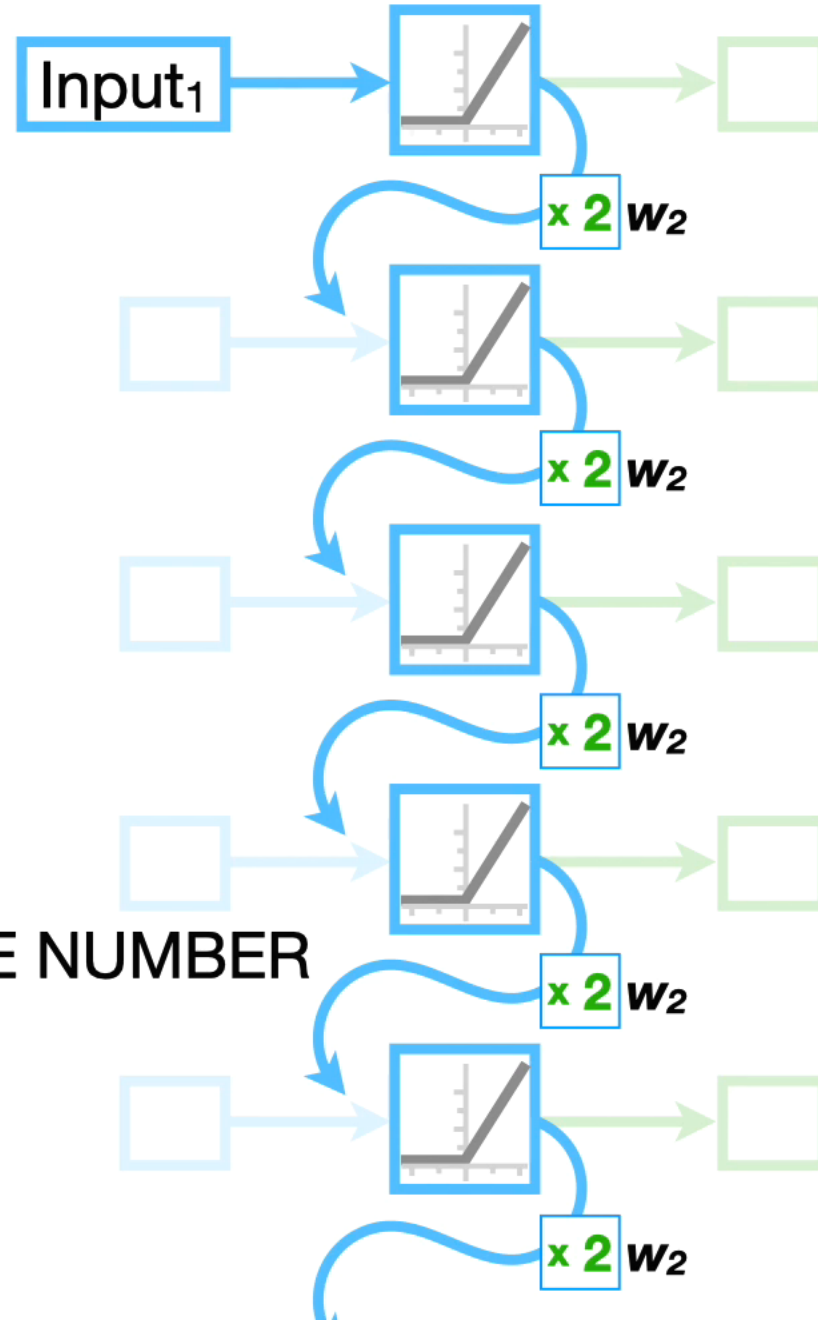


And this **HUGE NUMBER**
is why they call this an
Exploding Gradient
Problem.

KABOOM!!!

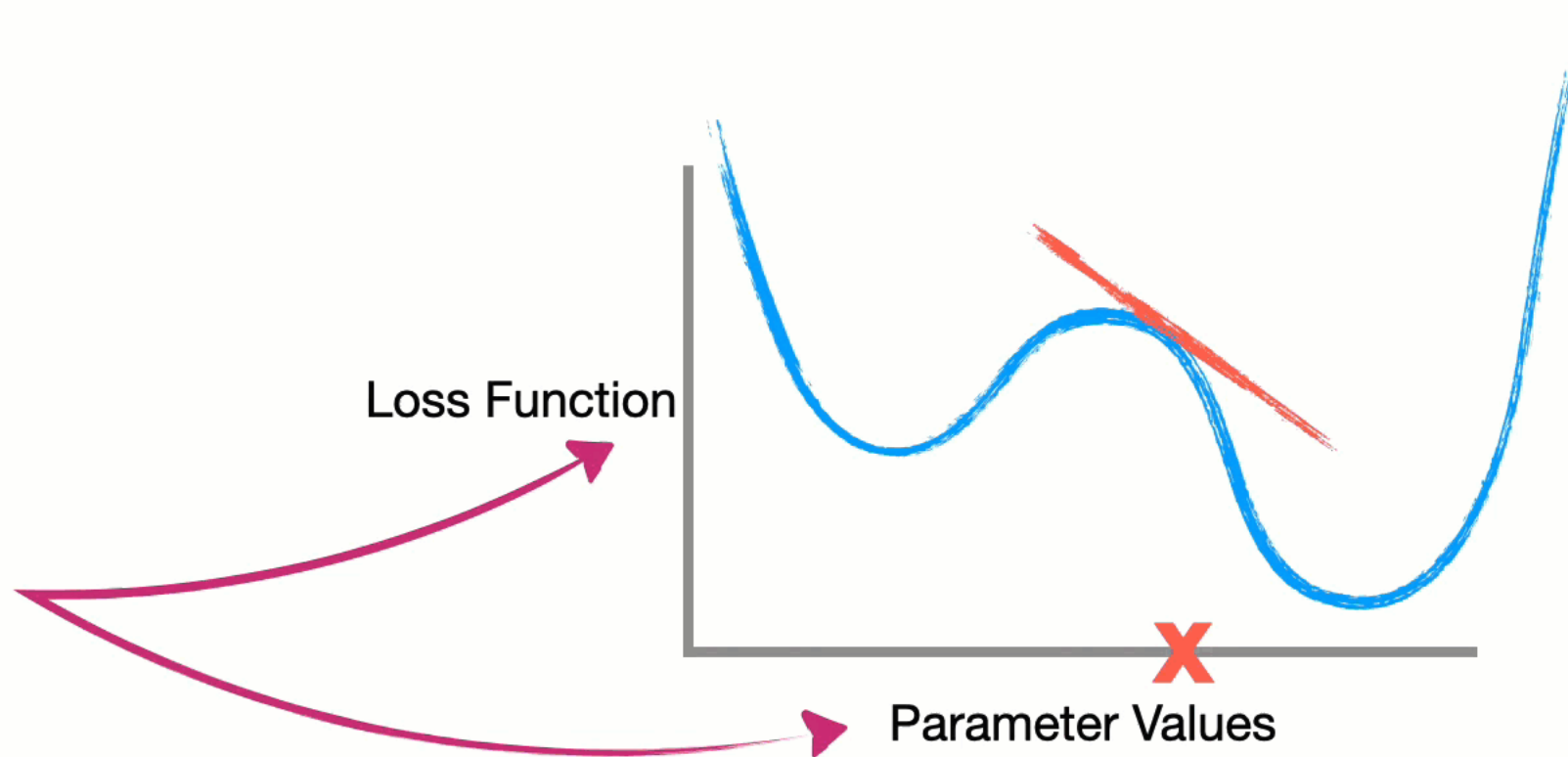
= $\text{Input}_1 \times \text{A HUGE NUMBER}$

= $\text{Input}_1 \times \text{A HUGE NUMBER}$



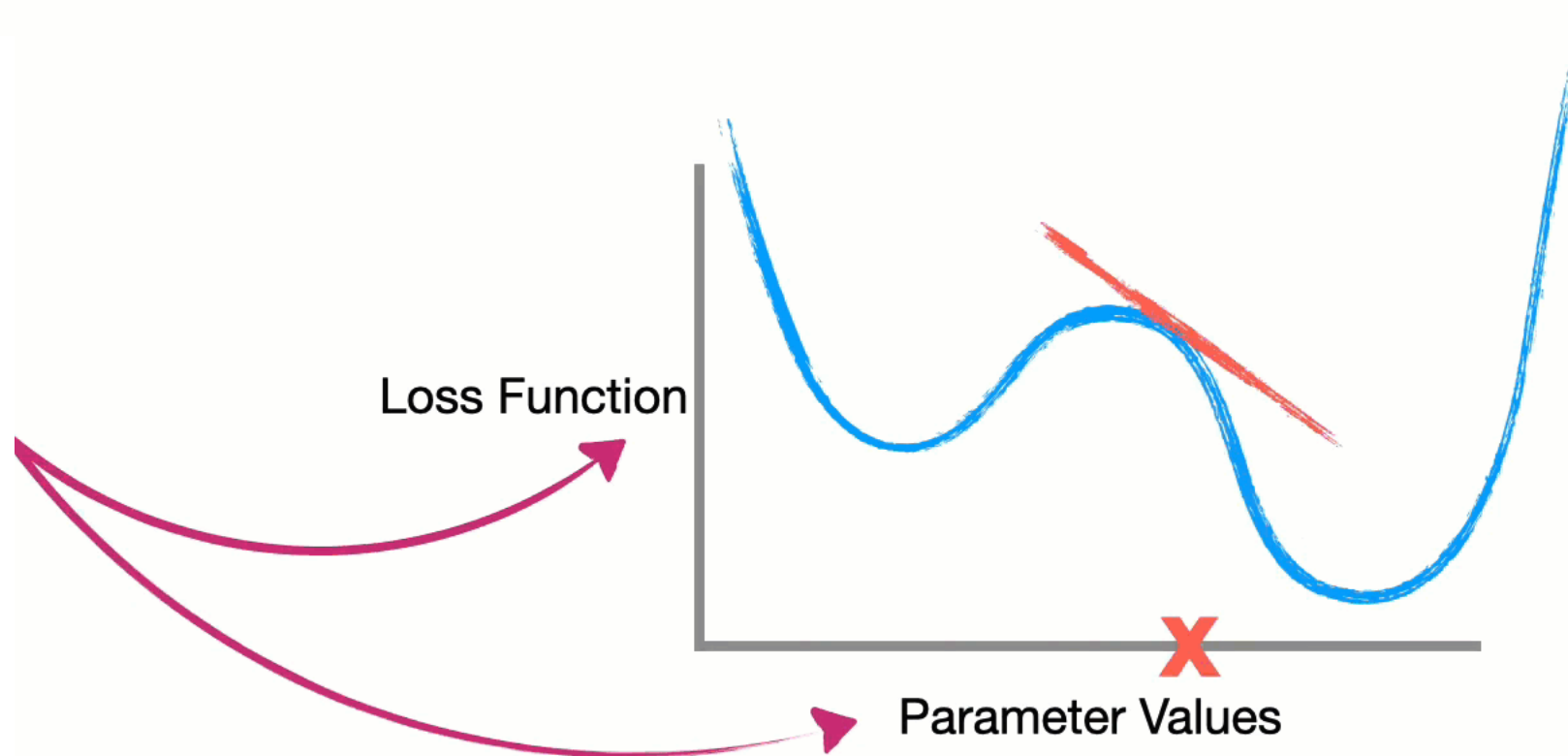


Normal Gradient



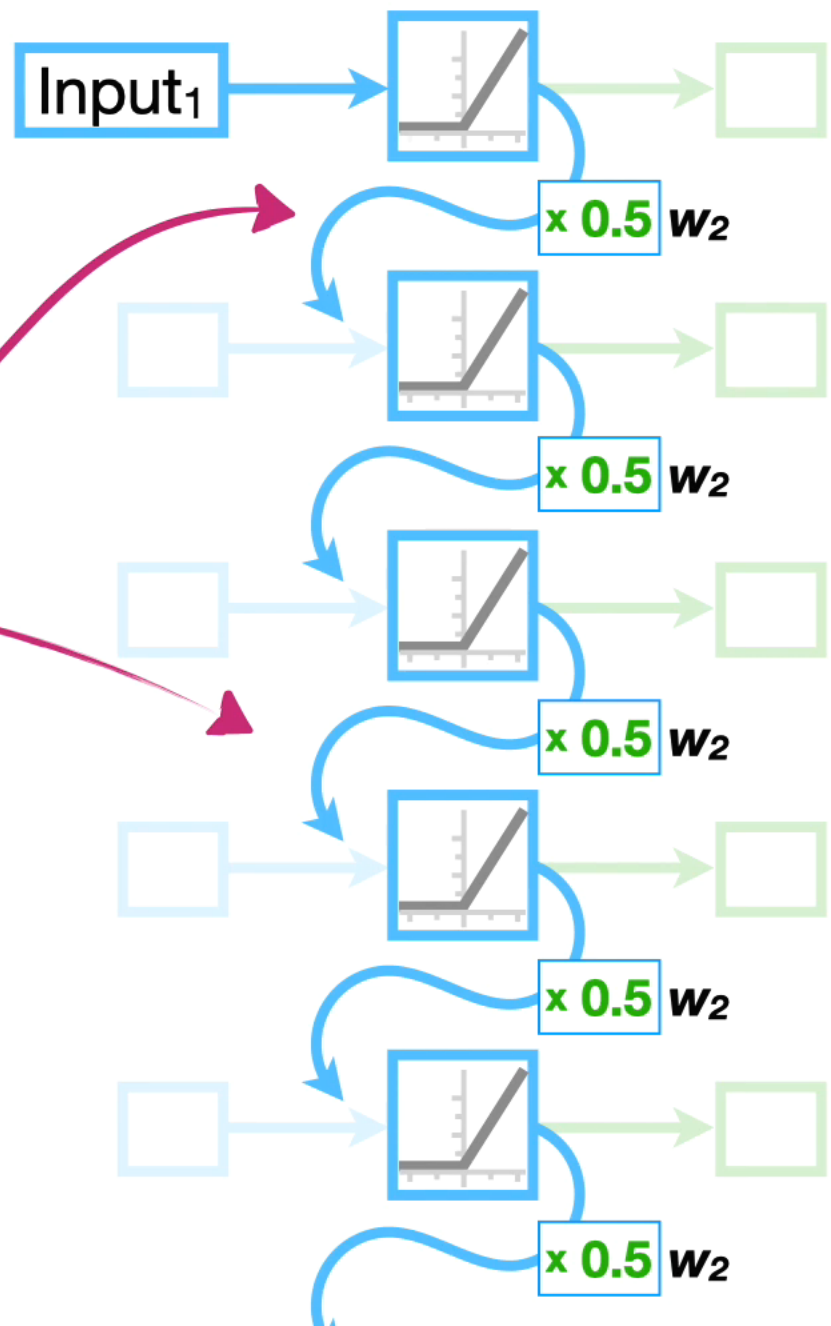


Gradient contains a Huge Number





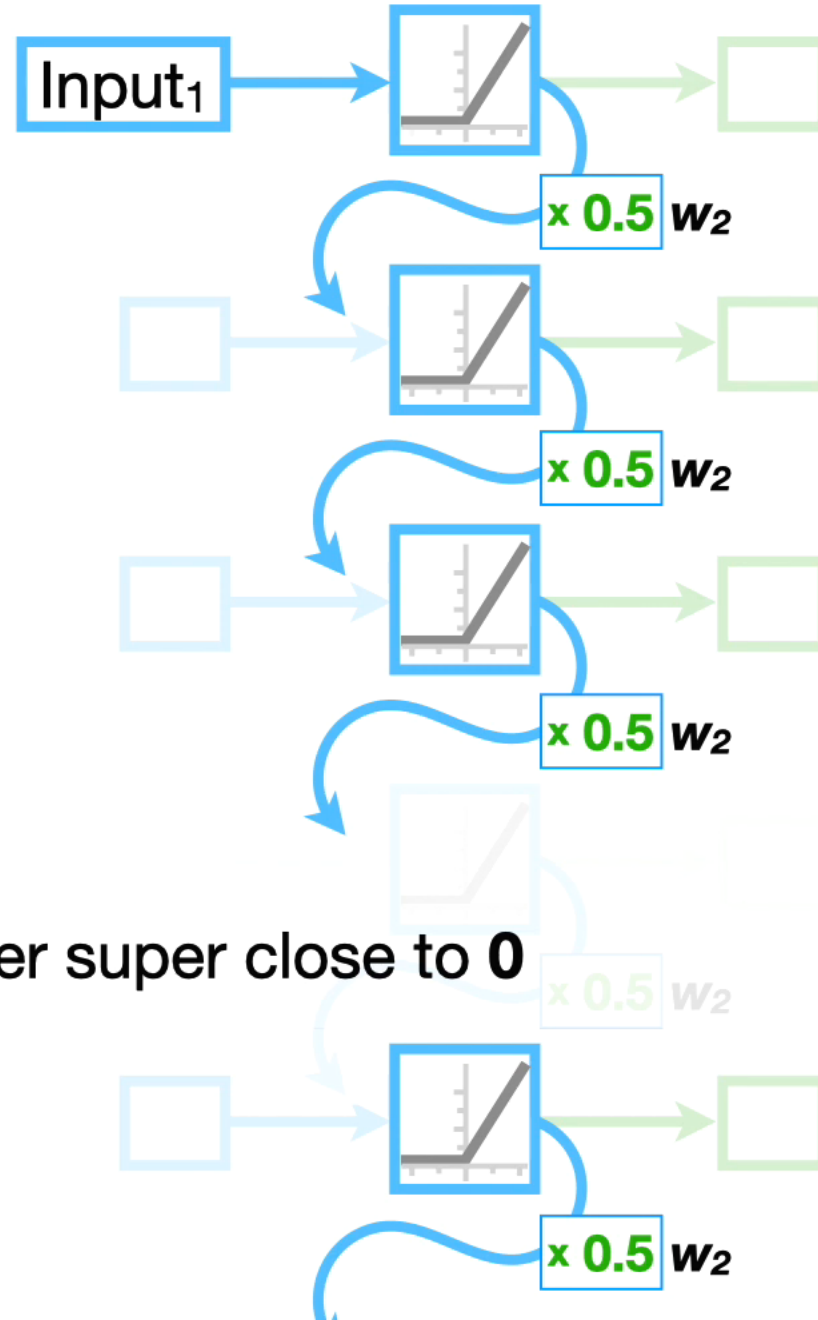
To illustrate **The Vanishing Gradient Problem**, let's set $w_2 = 0.5$.



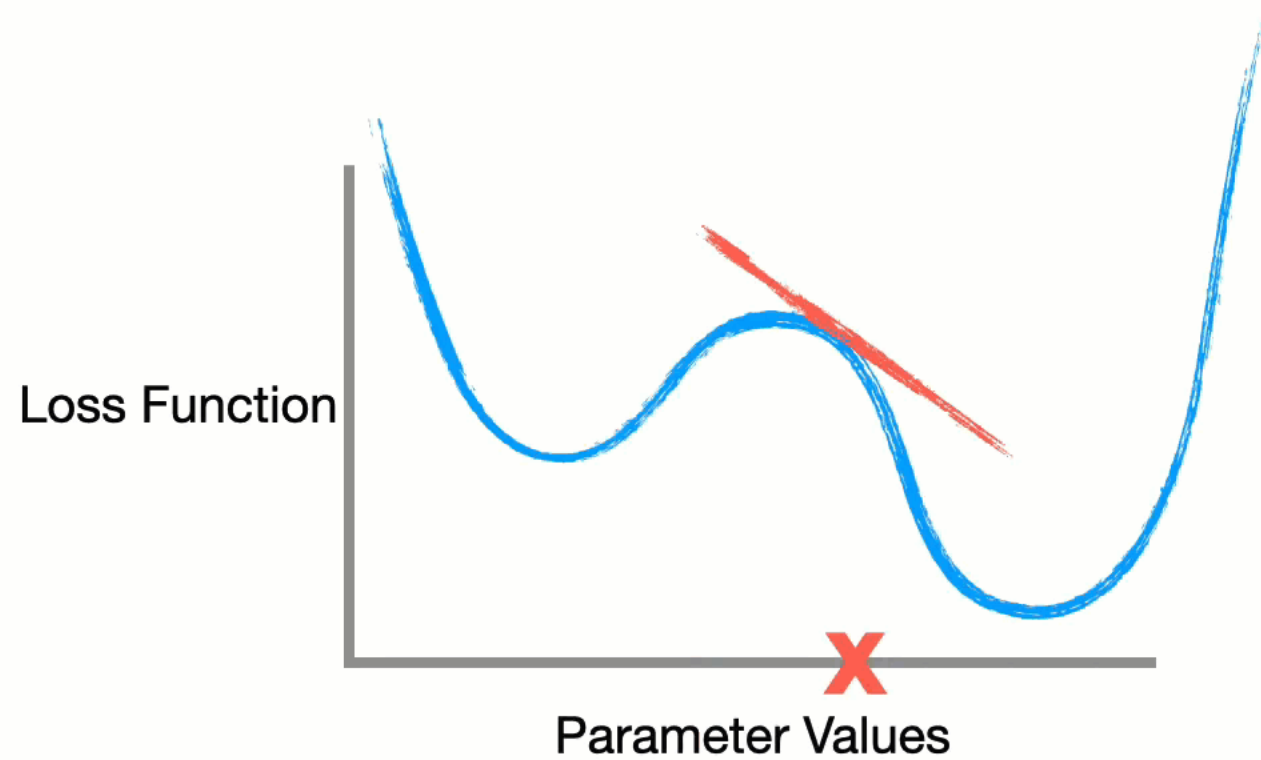
Because this number is **super close to 0**, this is called the **Vanishing Gradient Problem**.

$\text{Input}_1 \times 0.5^{50} = \text{Input}_1 \times \text{a number super close to } 0$

$\text{Input}_1 \times w_2^{\text{Num. Unroll}}$



**Gradient is
Too Small**



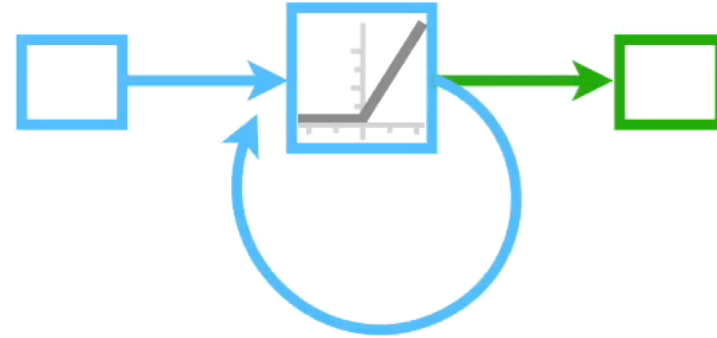


In summary, basic, vanilla
Recurrent Neural Networks
are hard to train because the
gradients can **explode**, or
vanish.

$$= \text{Input}_1 \times w_2^{\text{Num. Unroll}}$$



Long Short-Term Memory

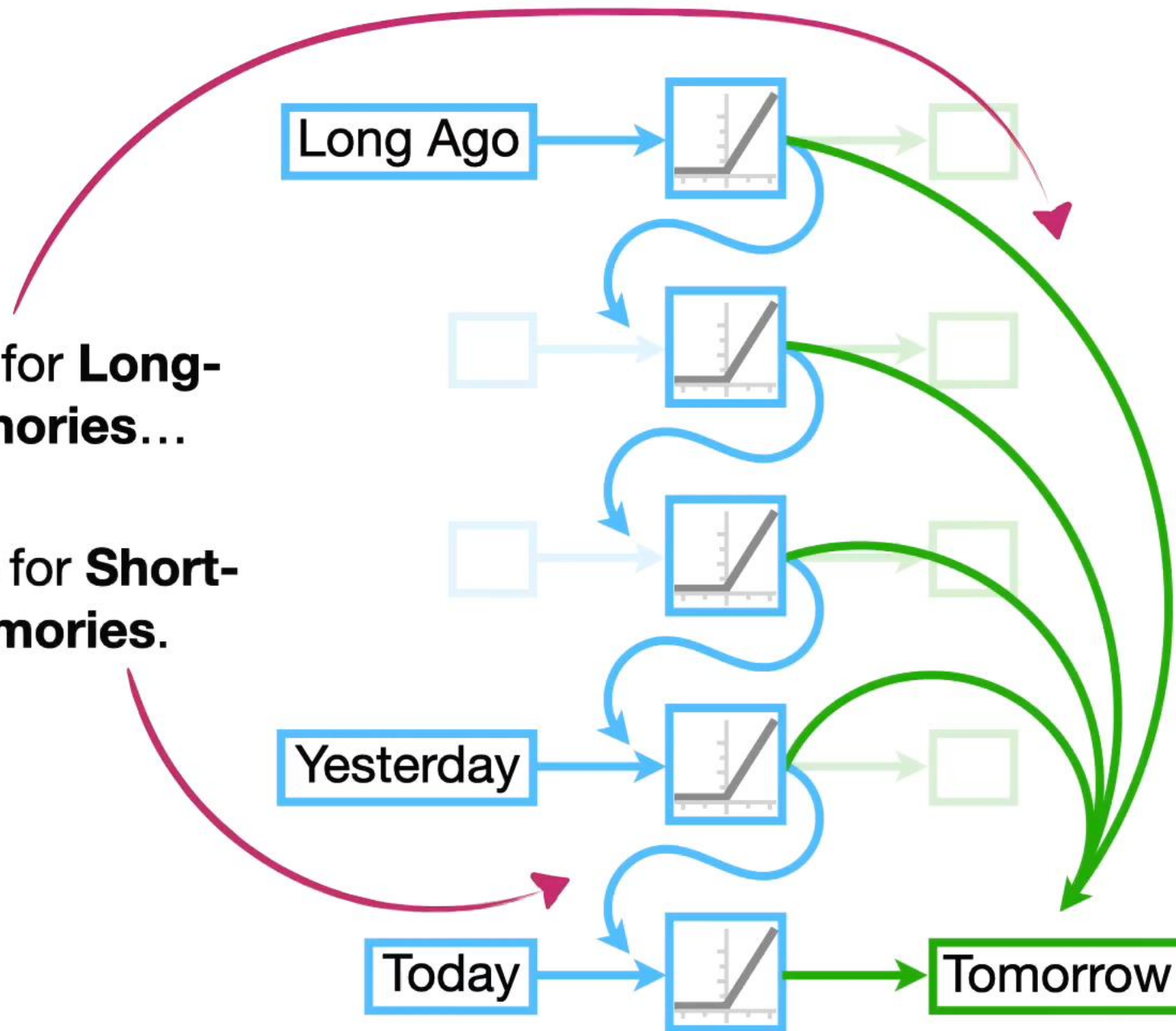


...is that instead of using the
same feedback loop
connection...

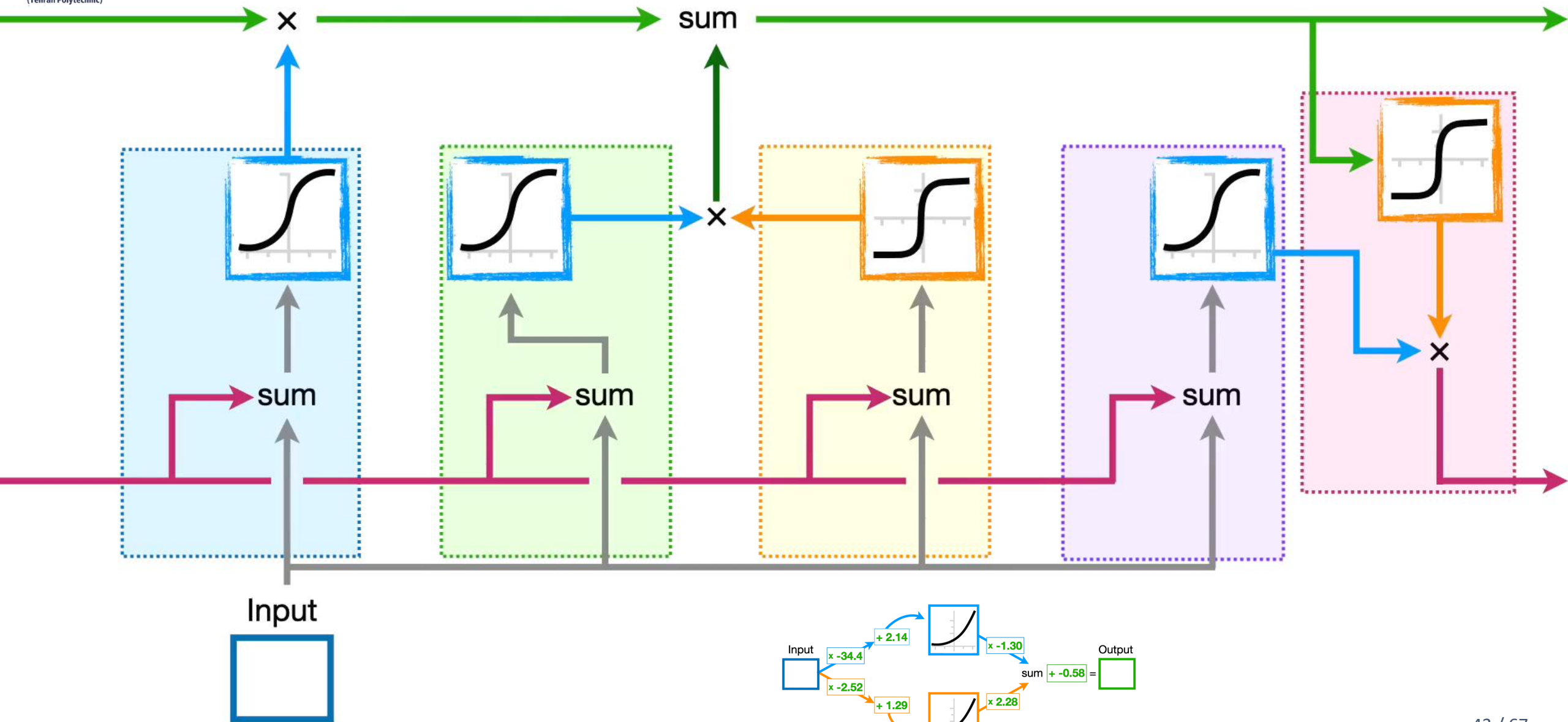




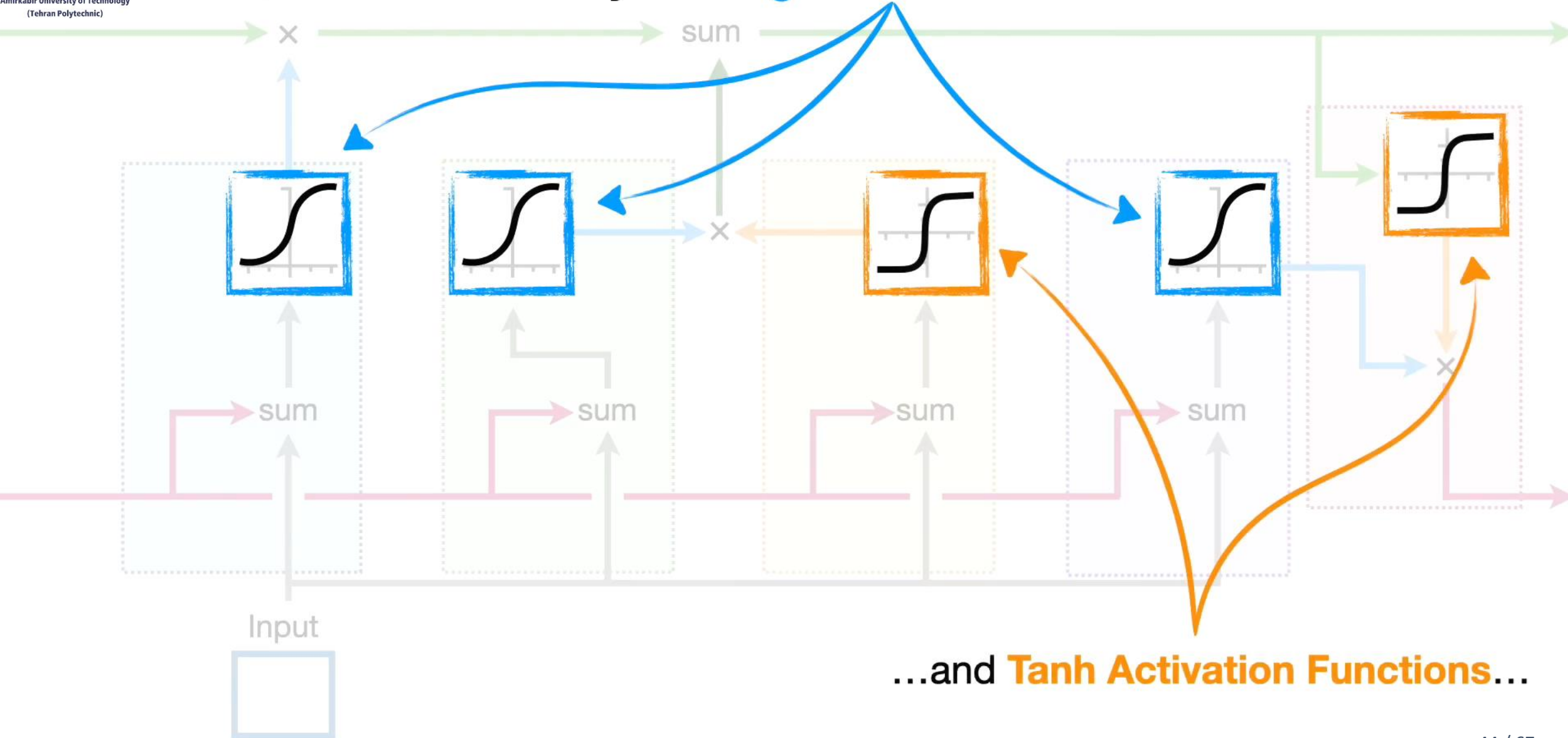
...and one is for **Short-Term Memories.**



...**Long Short-Term Memory** is based on a much more complicated unit.



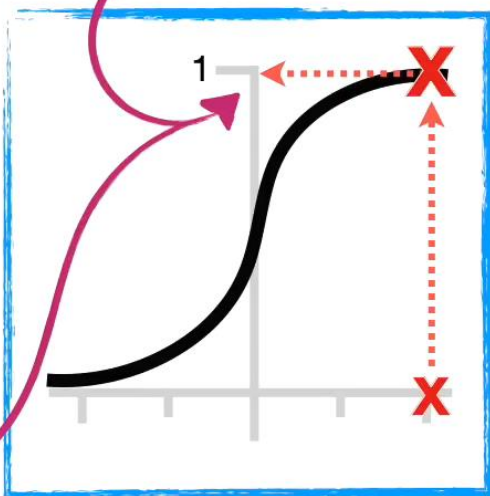
NOTE: Unlike the networks we've used before in this series, **Long Short-Term Memory** uses **Sigmoid Activation Functions**...





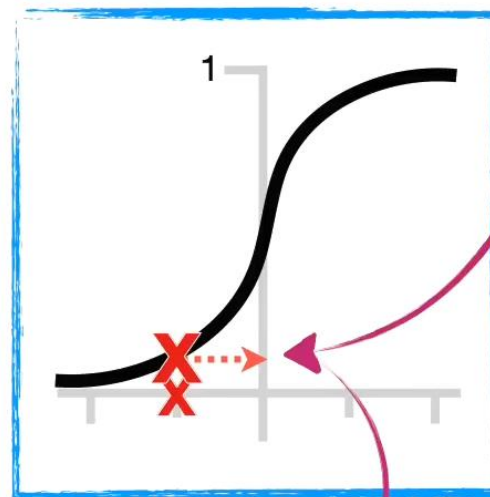
...we get **0.99995** as
the y-axis coordinate.

$$f(x) = \frac{e^x}{e^x + 1}$$
$$f(10) = \frac{e^{10}}{e^{10} + 1}$$
$$= 0.99995$$



...then we get **0.01** as
the y-axis coordinate.

$$f(x) = \frac{e^x}{e^x + 1}$$
$$f(-5) = \frac{e^{-5}}{e^{-5} + 1}$$
$$= 0.01$$



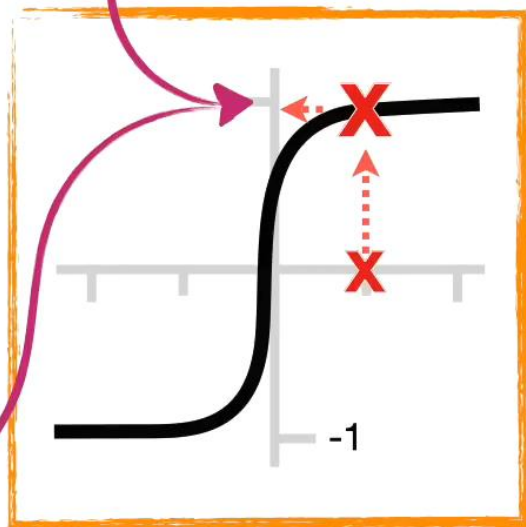


...we get **0.96** as the
y-axis coordinate.

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$f(2) = \frac{e^2 - e^{-2}}{e^2 + e^{-2}}$$

$$= 0.96$$

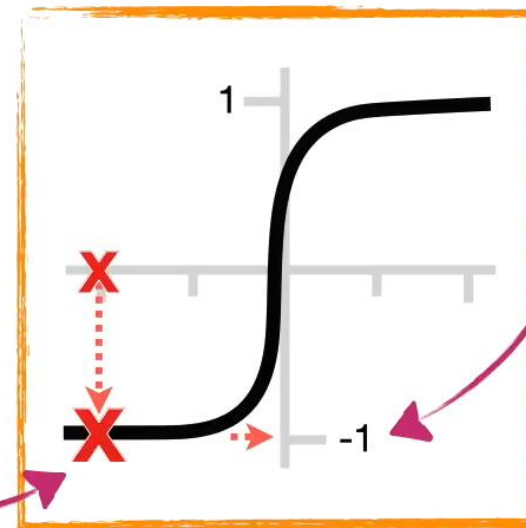


...we get **-1** as the
y-axis coordinate.

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$f(-5) = \frac{e^{-5} - e^5}{e^{-5} + e^5}$$

$$= -1$$





The lack of **Weights** allows the **Long-Term Memories** to flow through a series of unrolled units without causing the gradient to **explode** or **vanish**.

the **Cell State**

Input

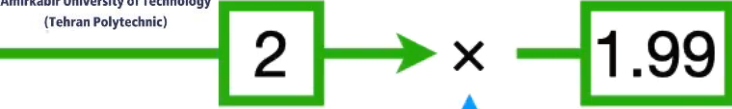




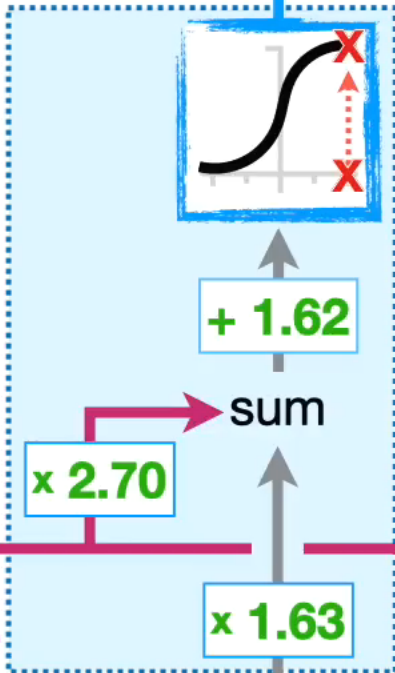
And, as we can see, the **Short-Term Memories** are directly connected to **Weights** that can modify them.



Long-Term Memory

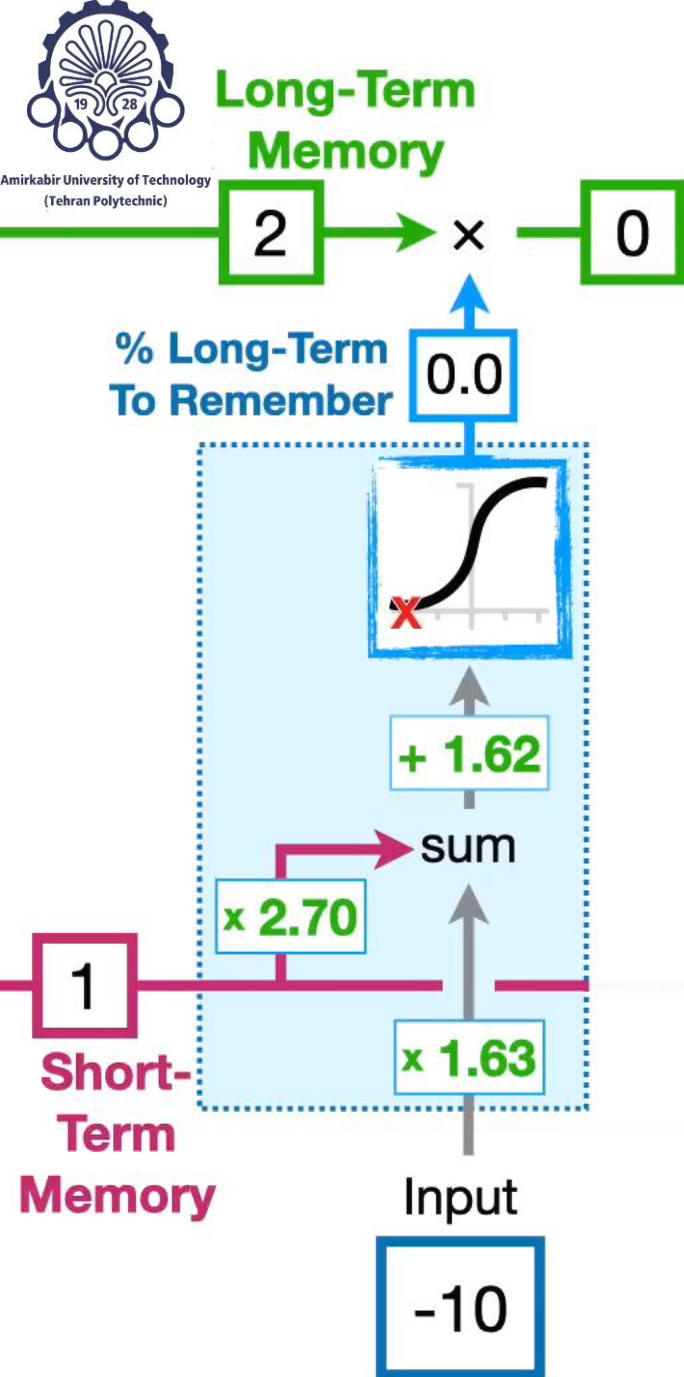


0.997



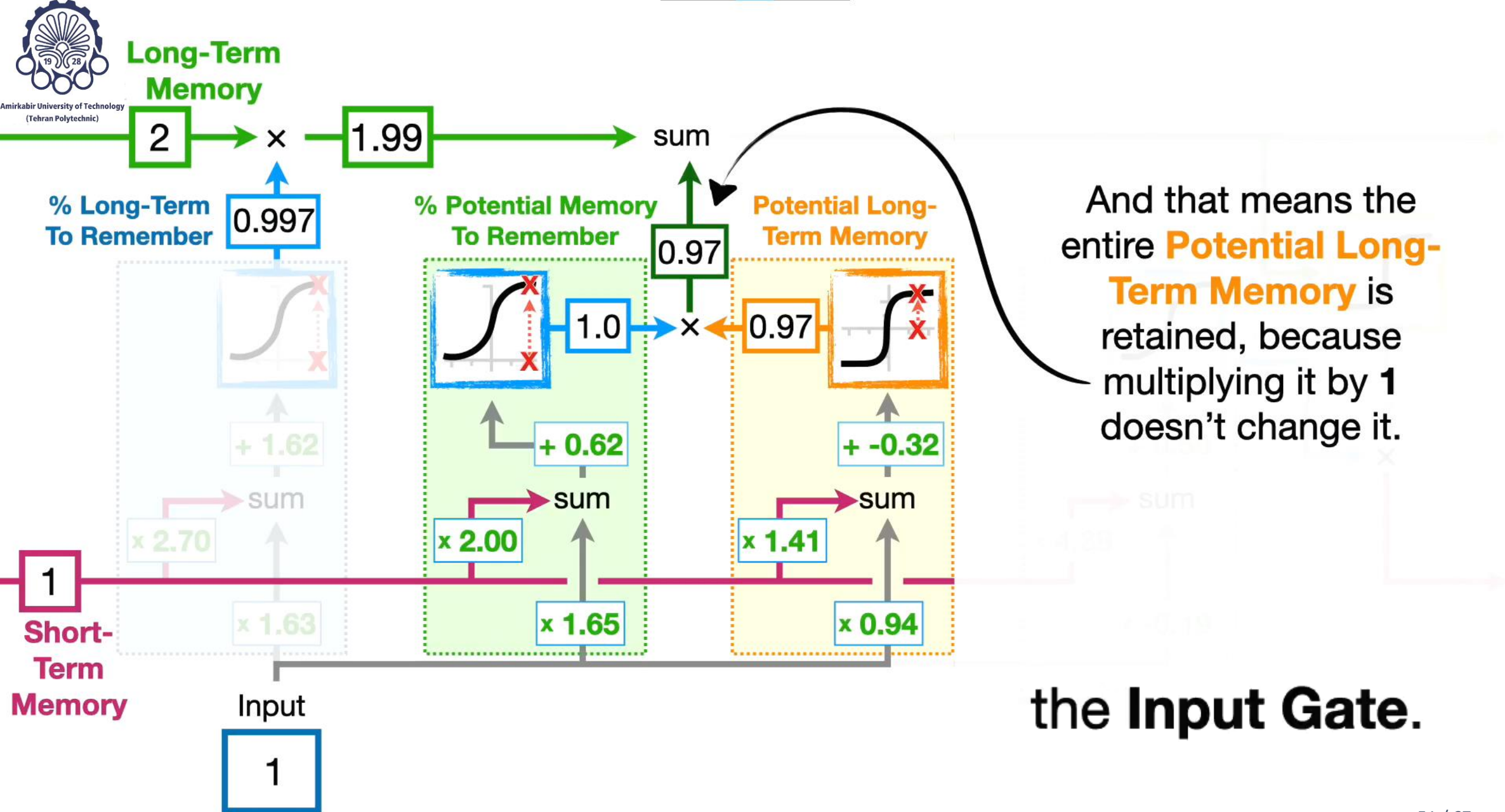
So this first stage of the **Long Short-Term Memory (LSTM)** unit reduced the **Long-Term Memory** by a little bit.

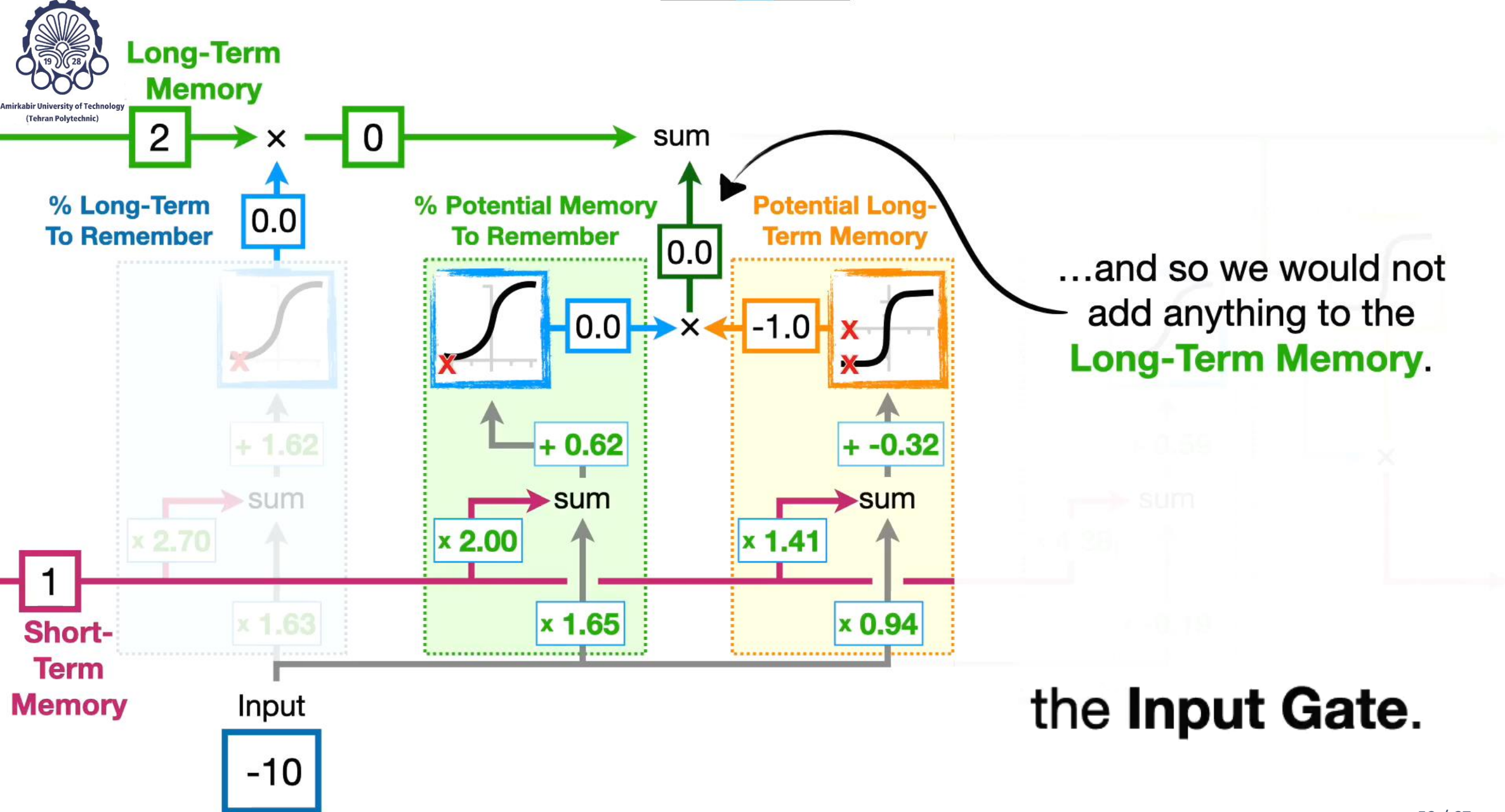
the **Forget Gate**.

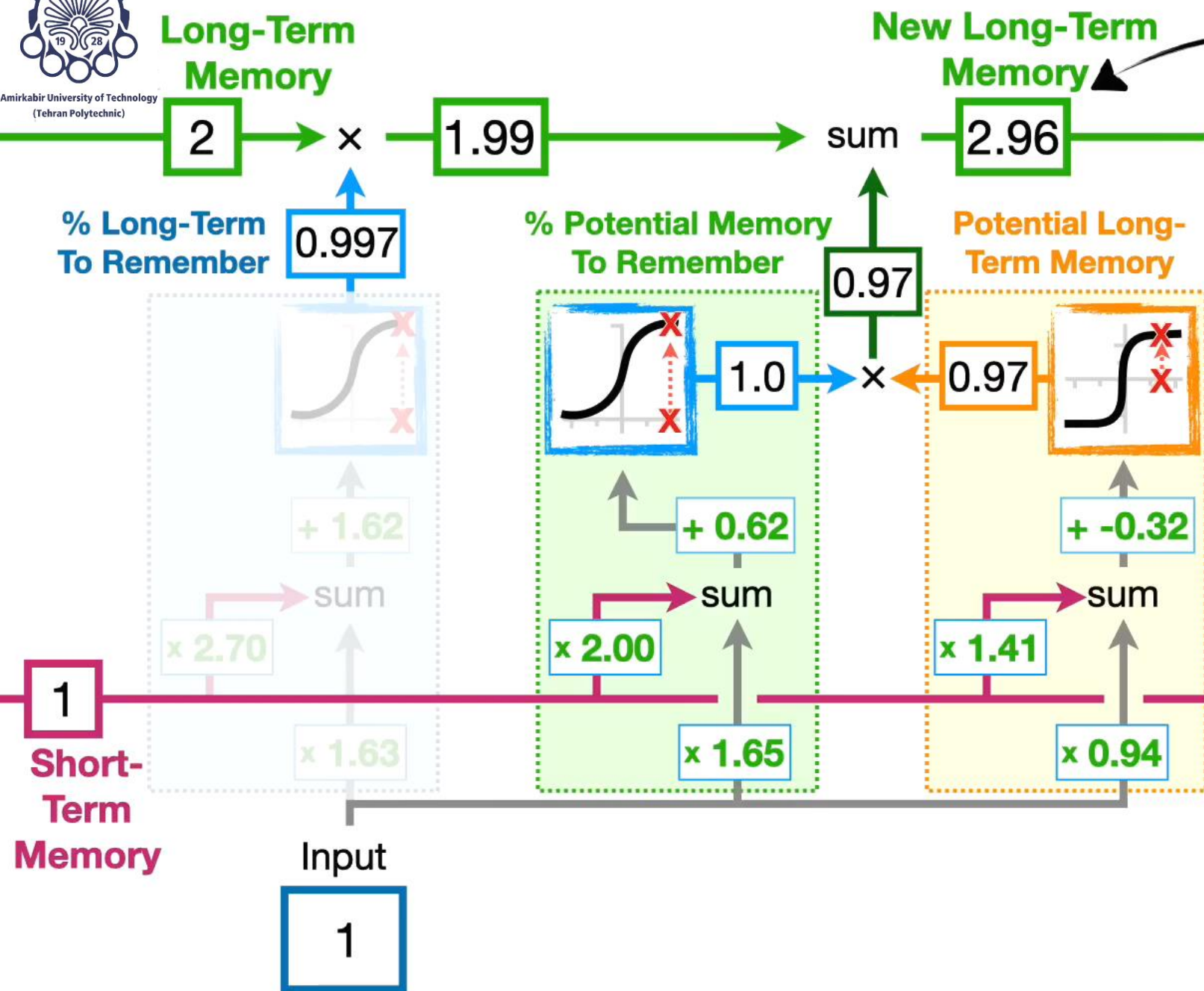


To summarize, the first stage in a **Long Short-Term Memory** unit determines what percentage of the **Long-Term Memory** is remembered.

the **Forget Gate**.



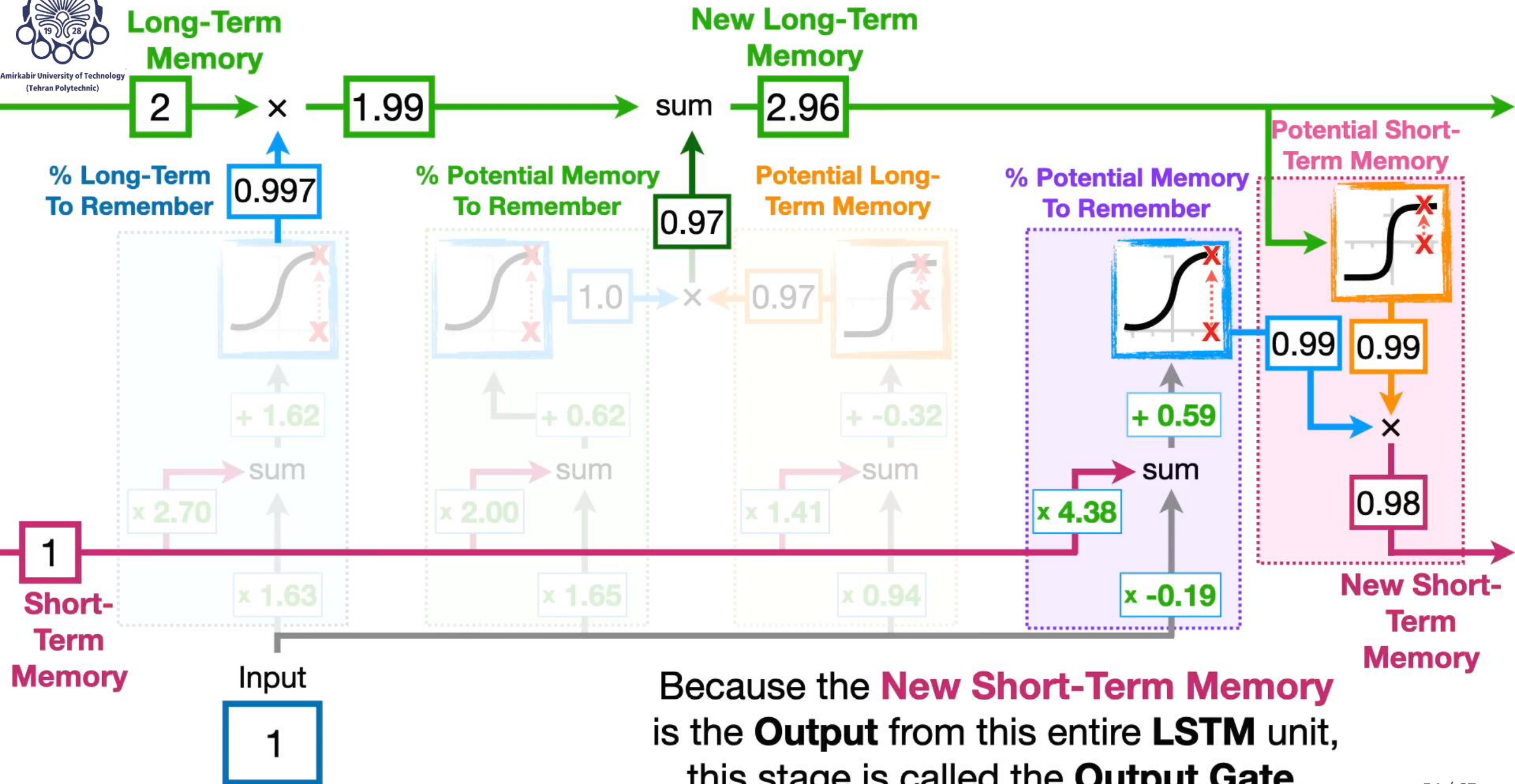


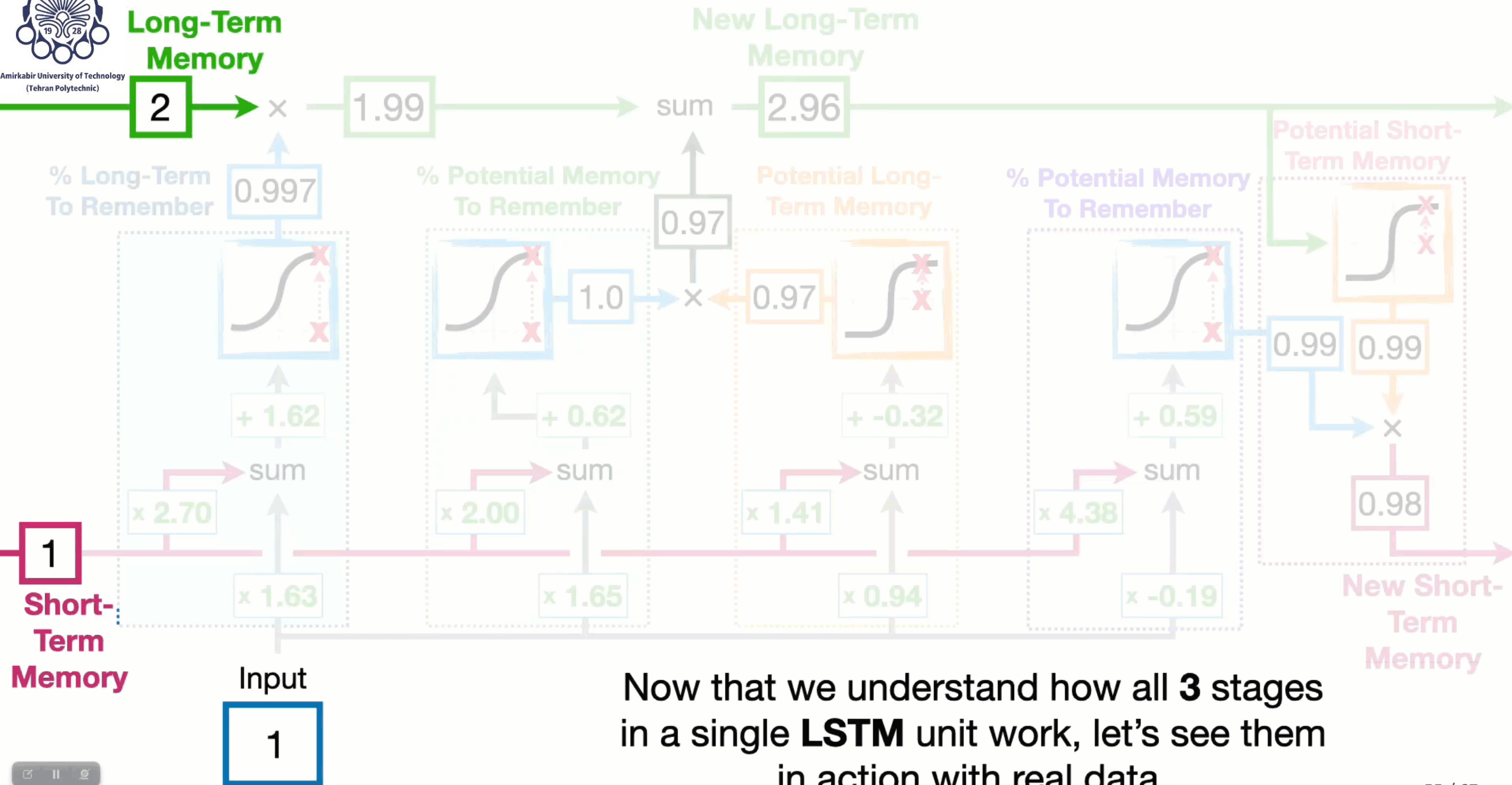


...and we get a new **Long-Term Memory**, 2.96.

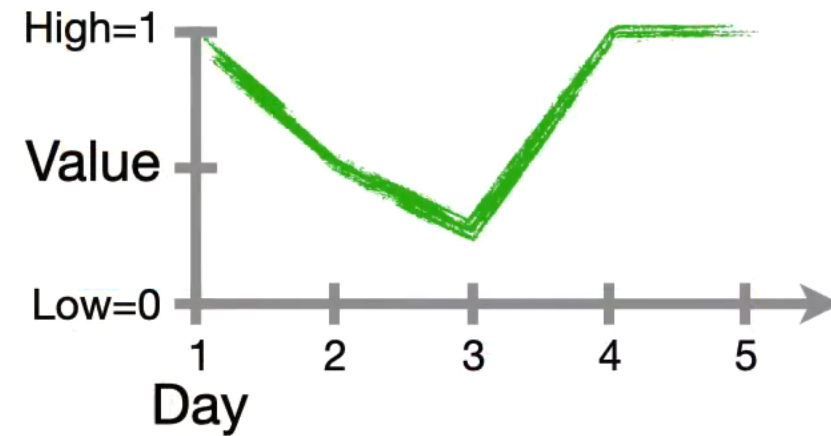
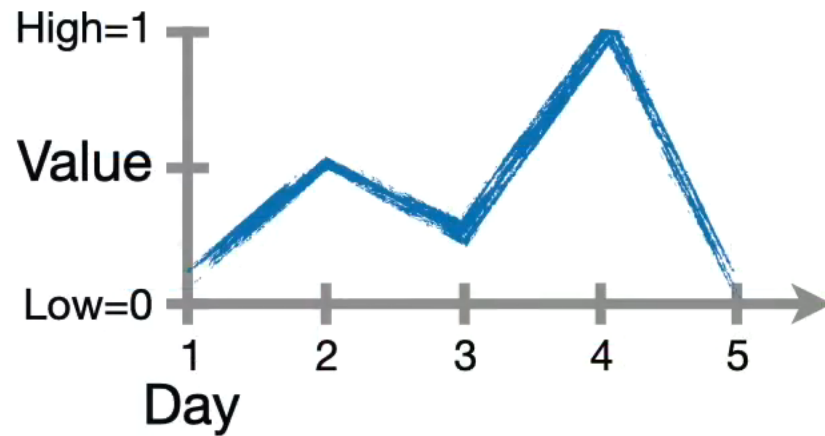
determines how we should update the **Long-Term Memory**...

the **Input Gate**.



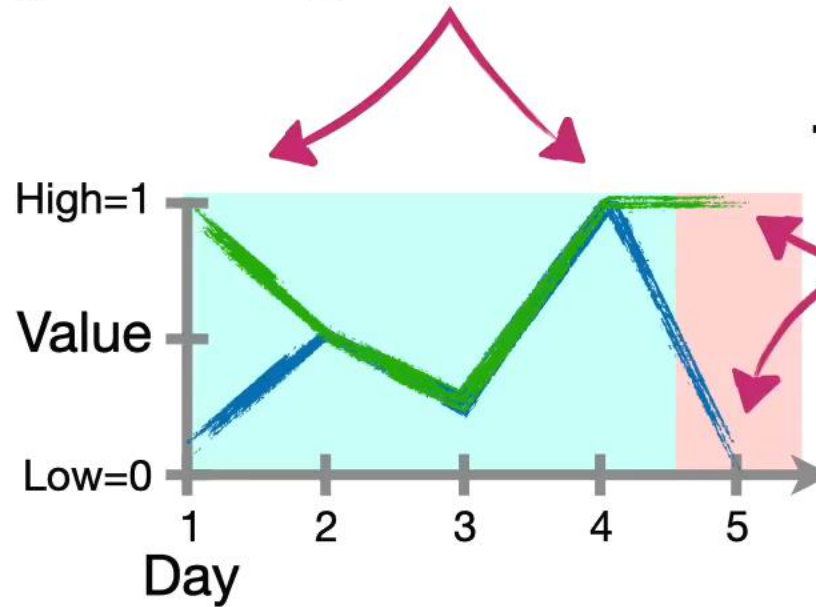


Here we have stock prices for two companies, **Company A** and **Company B**.



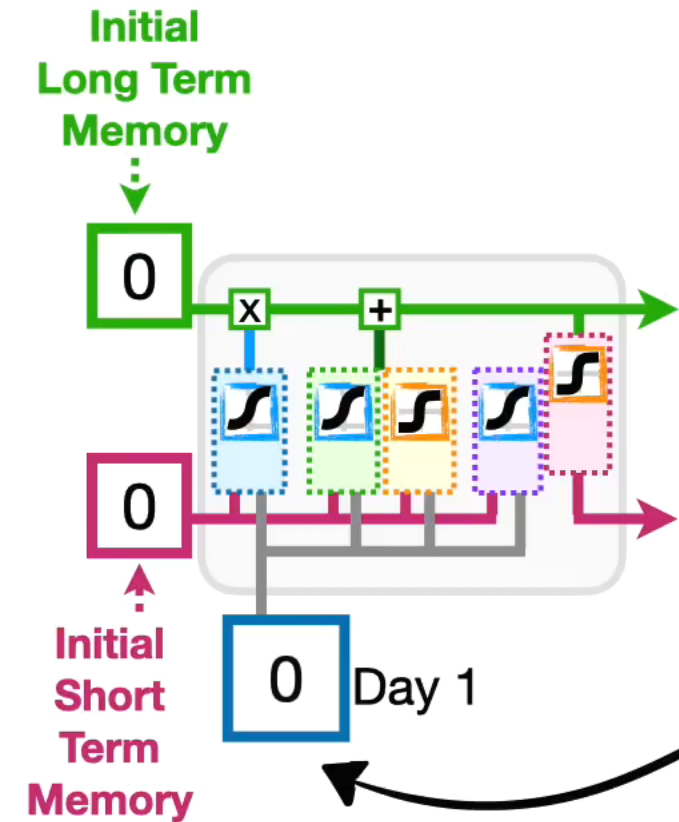
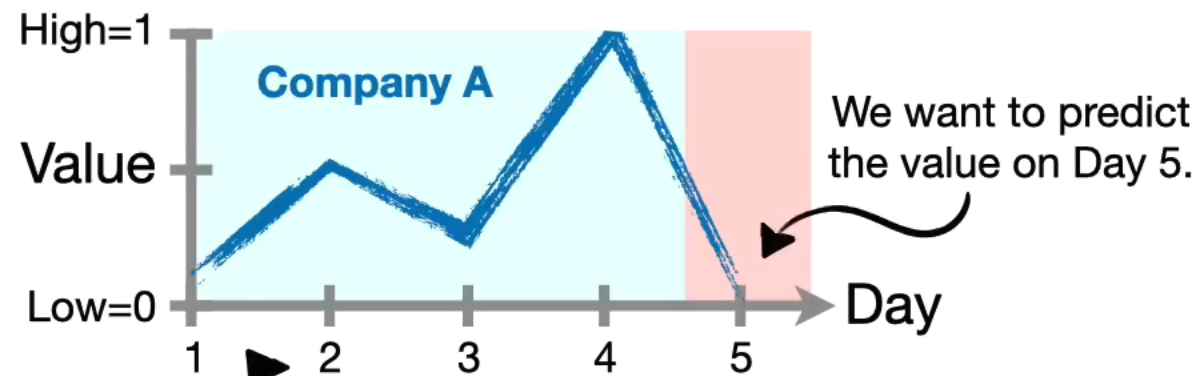
Company A = 
Company B = 

In other words, we're going to sequentially run the data from **Days 1** through **4** through an unrolled **LSTM**...

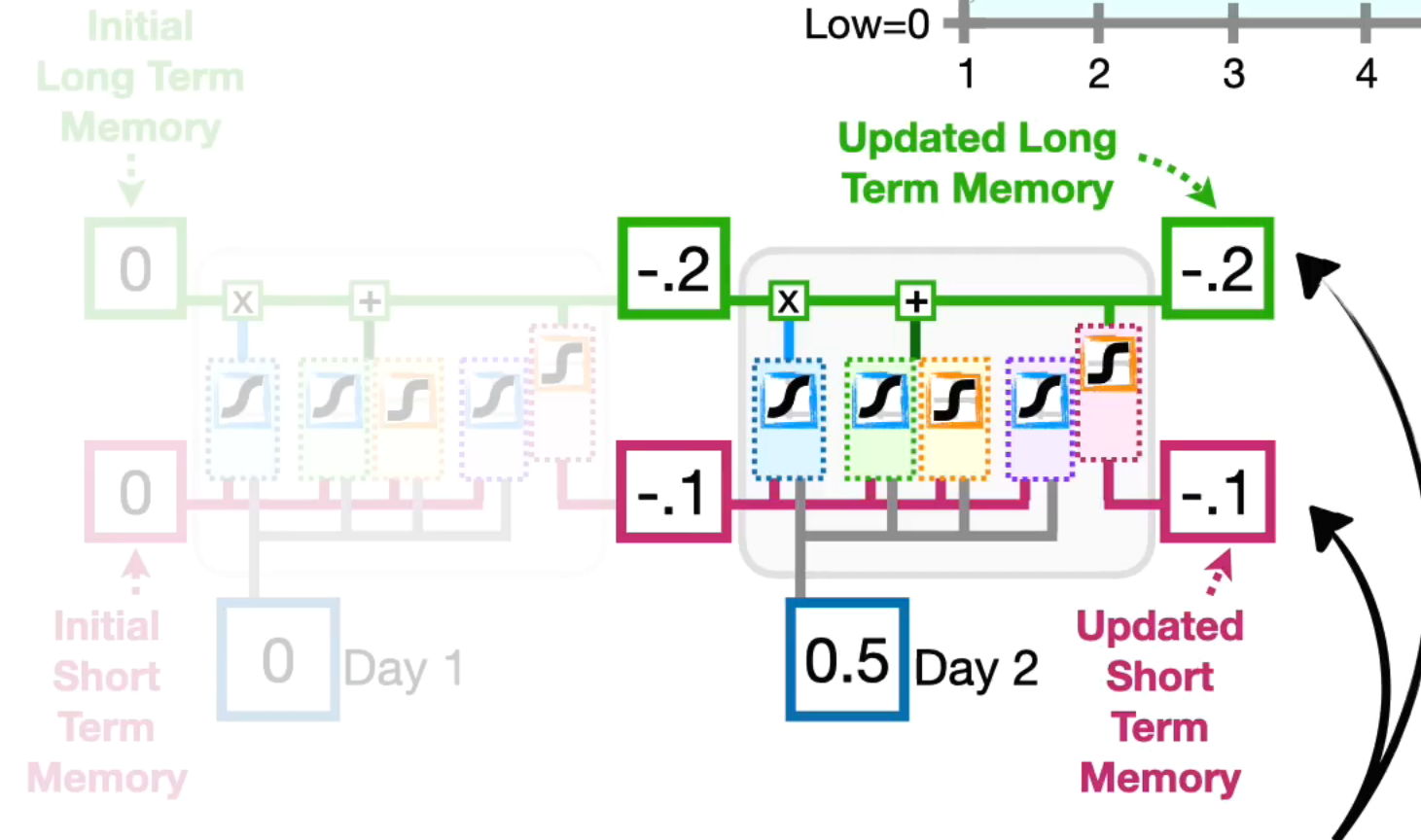
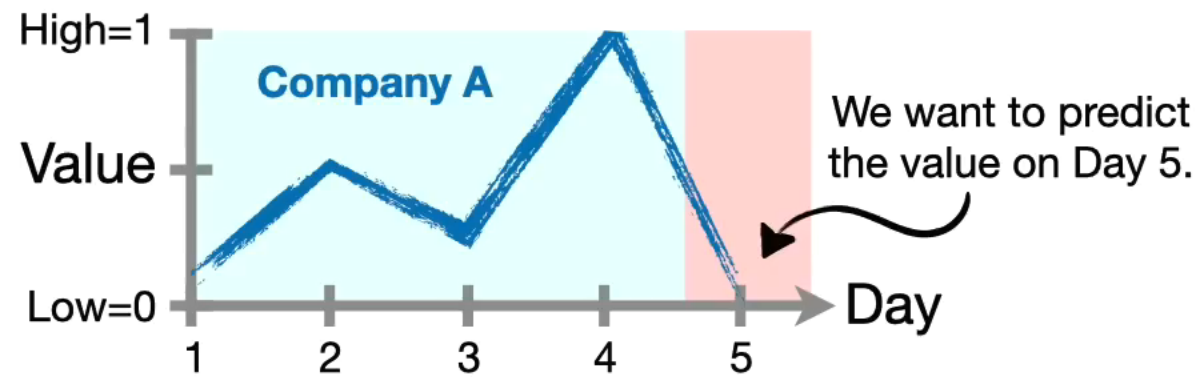


...and see if it can correctly predict the values for **Day 5** for both companies.

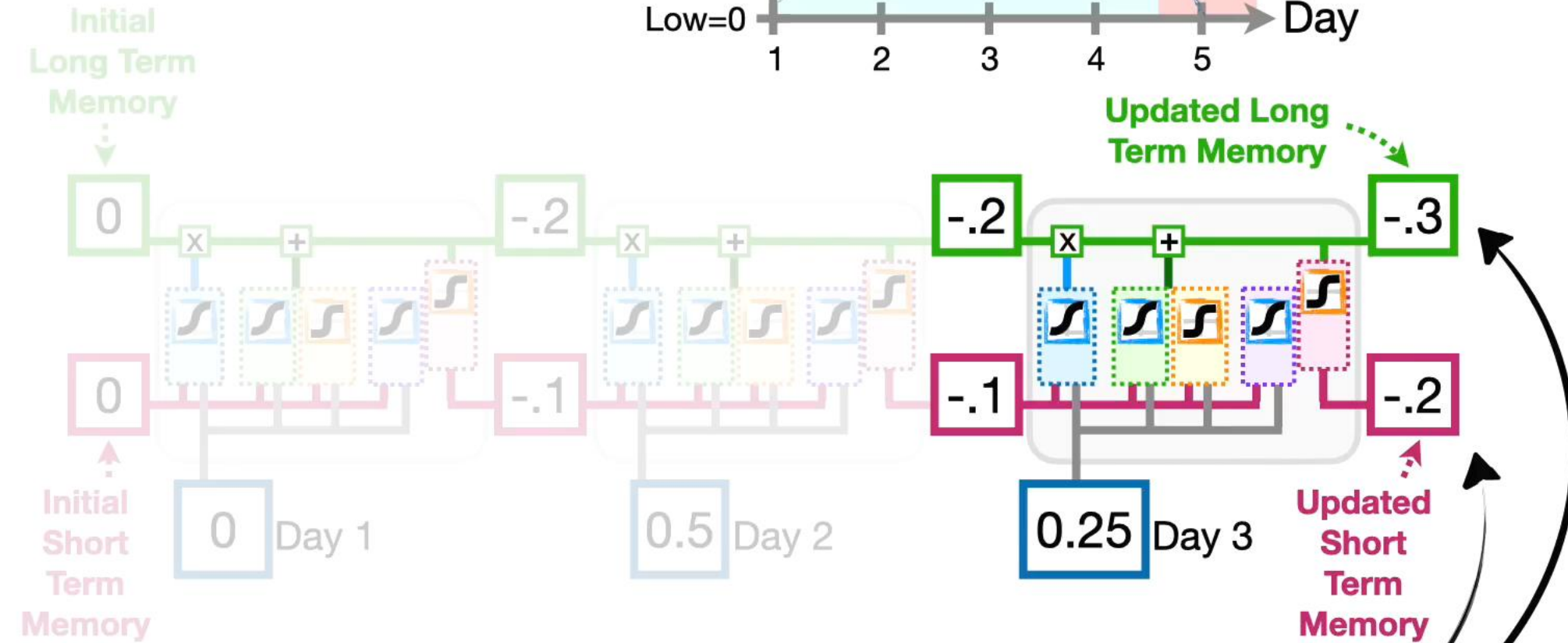
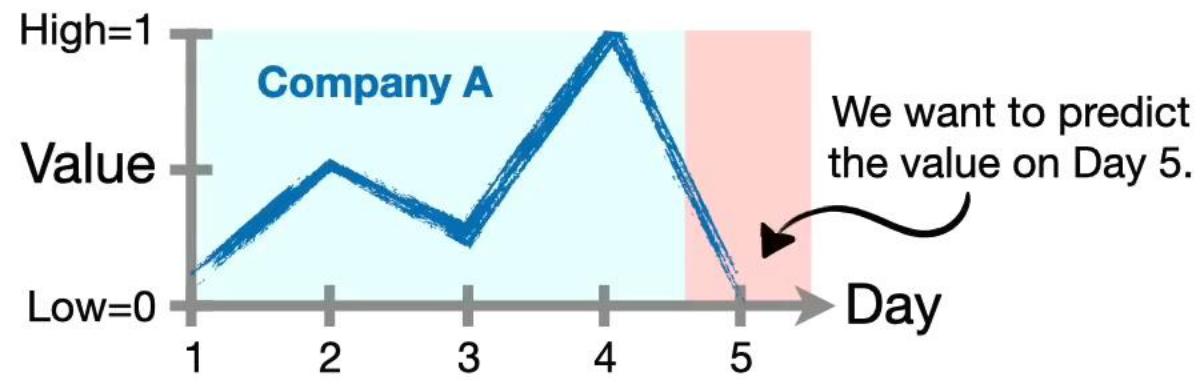
Company A = 
Company B = 



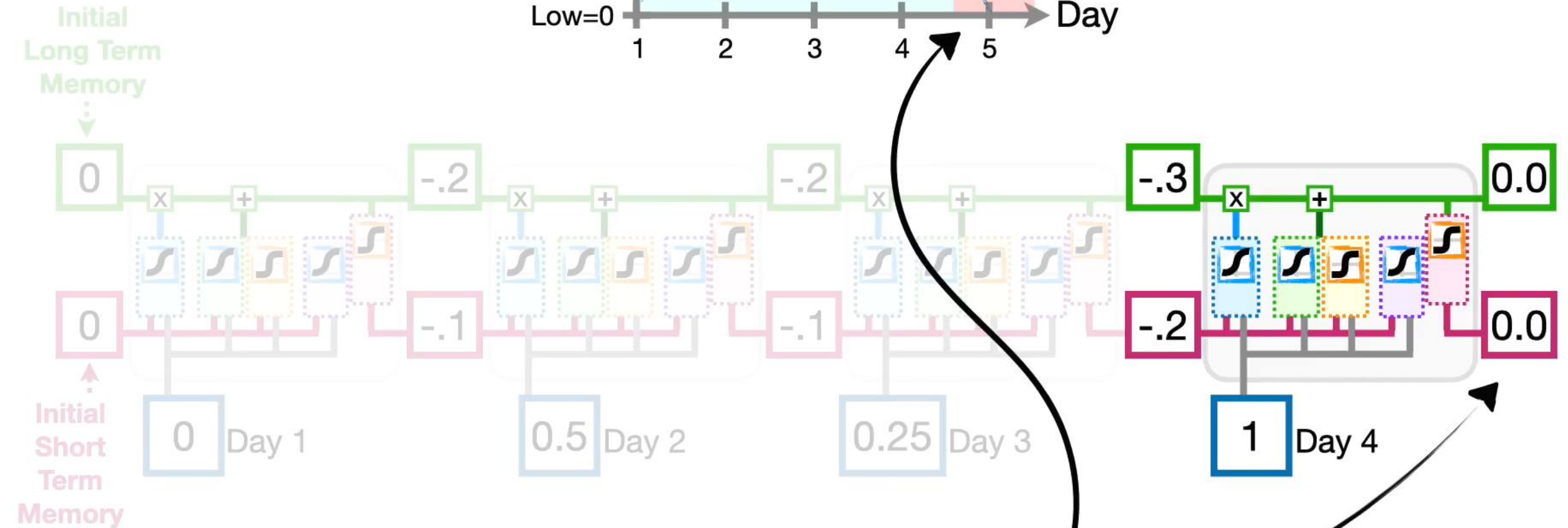
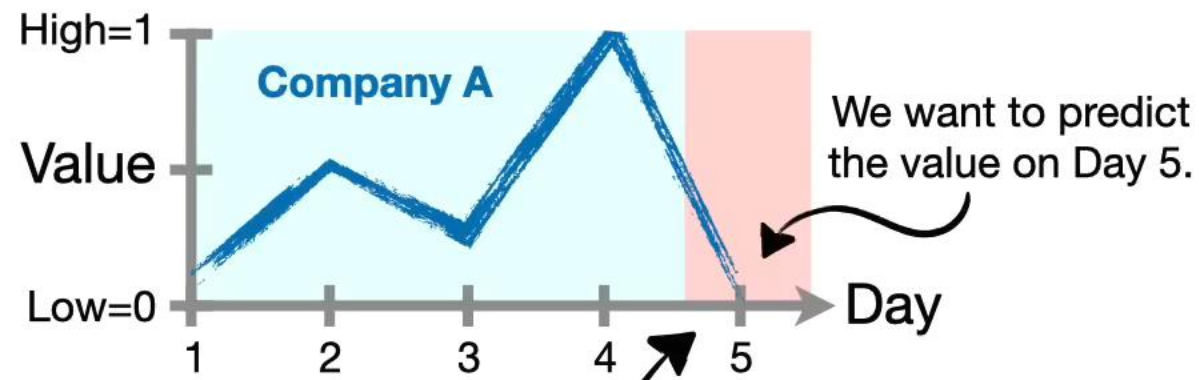
...then we'll start by plugging the value for **Day 1**, which is **0**, into the **Input**.



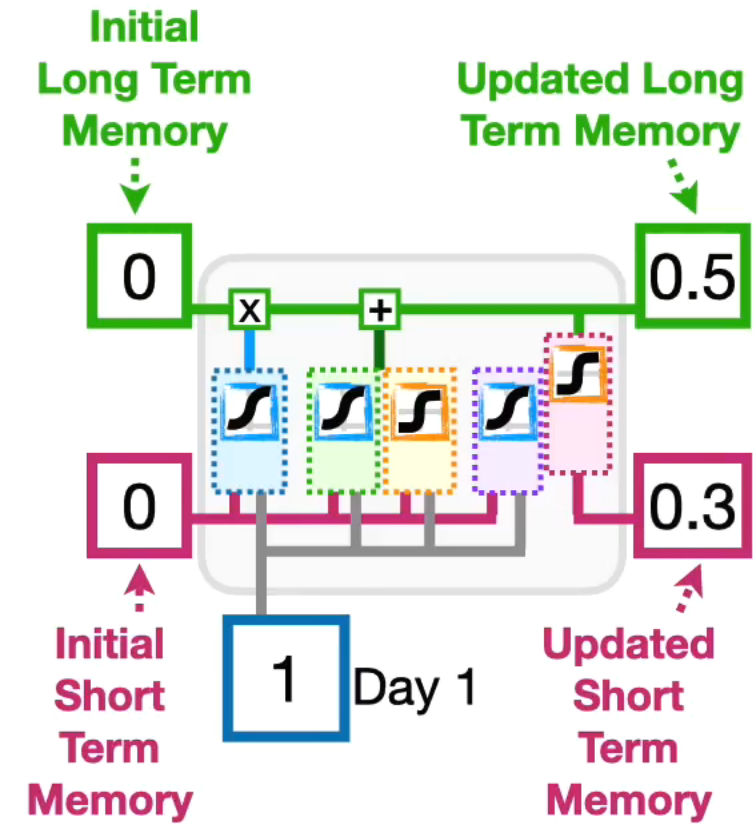
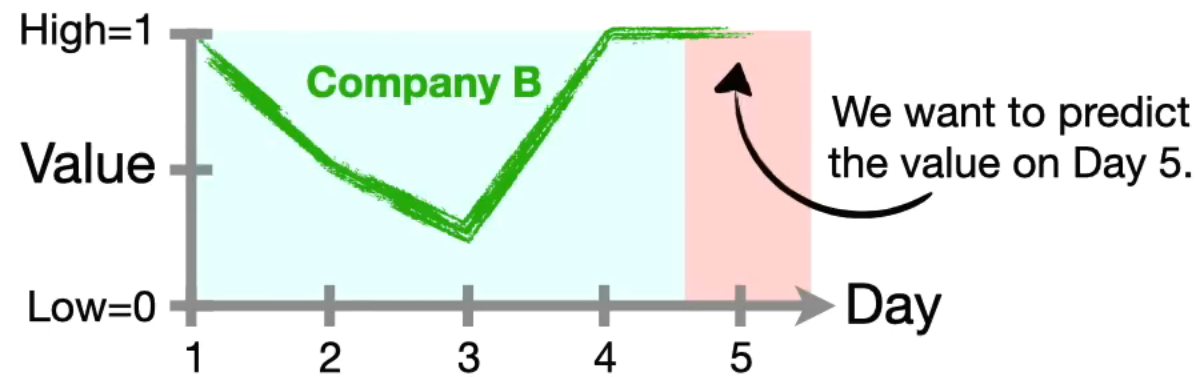
...and we end up with these updated
Long and Short-Term Memories.



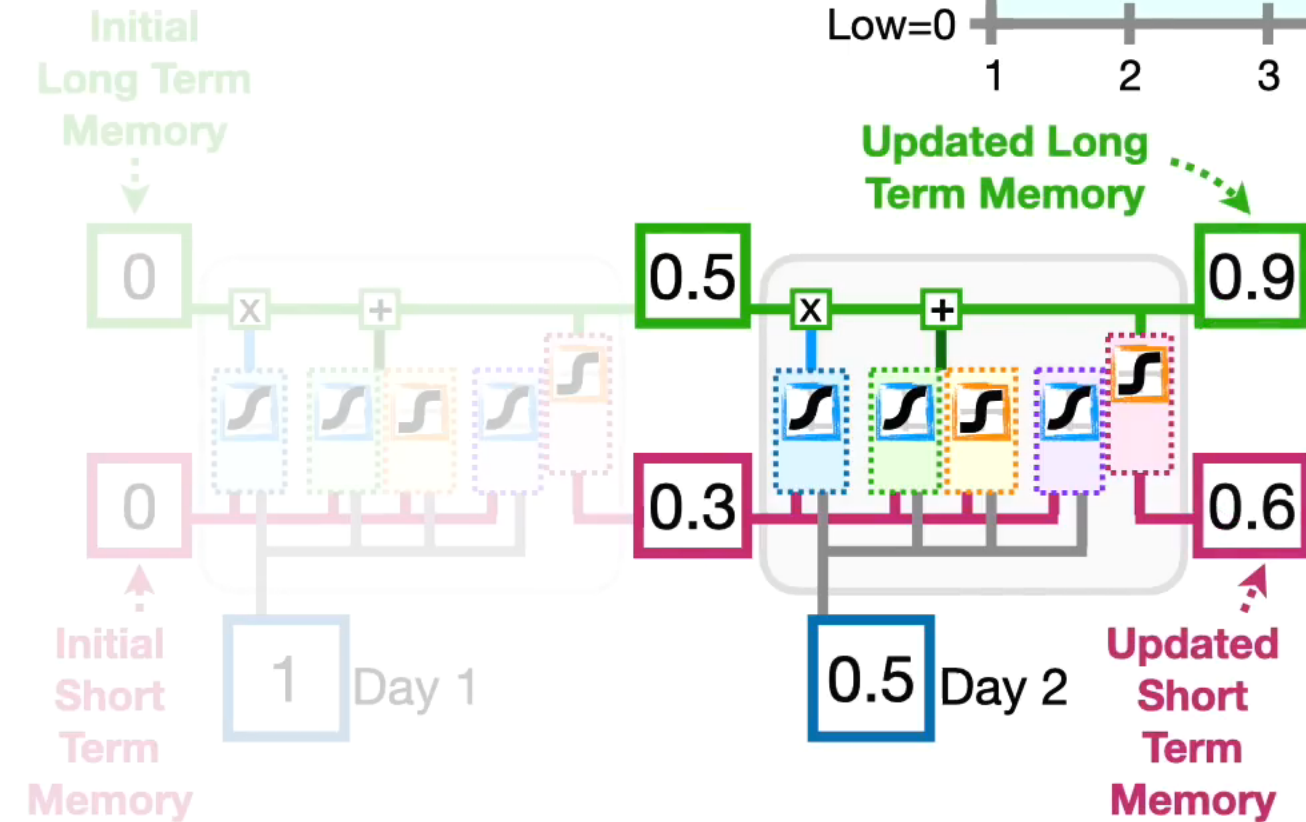
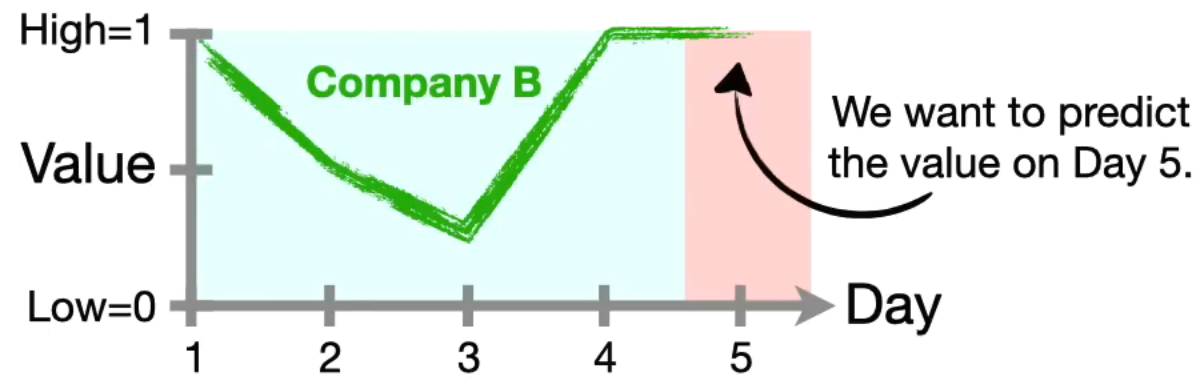
...and gives us these updated memories.



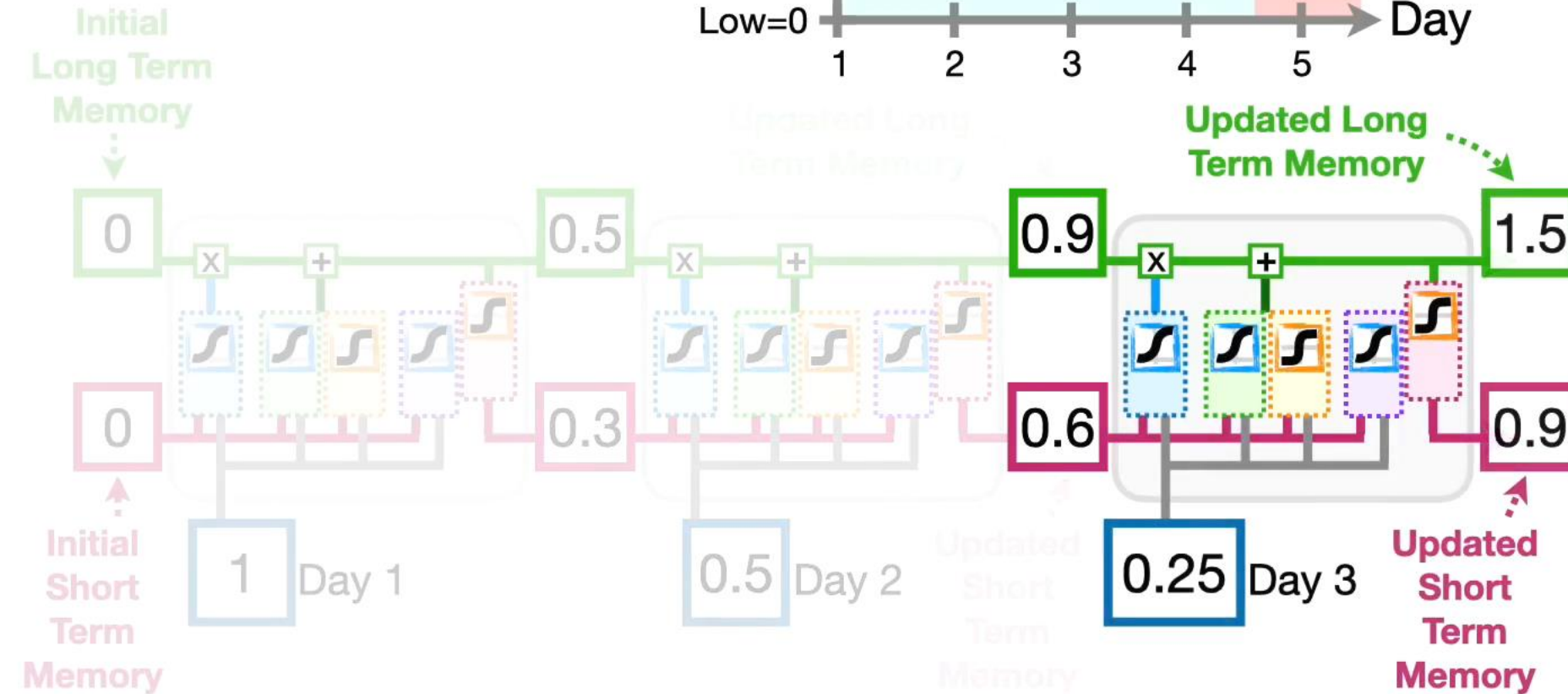
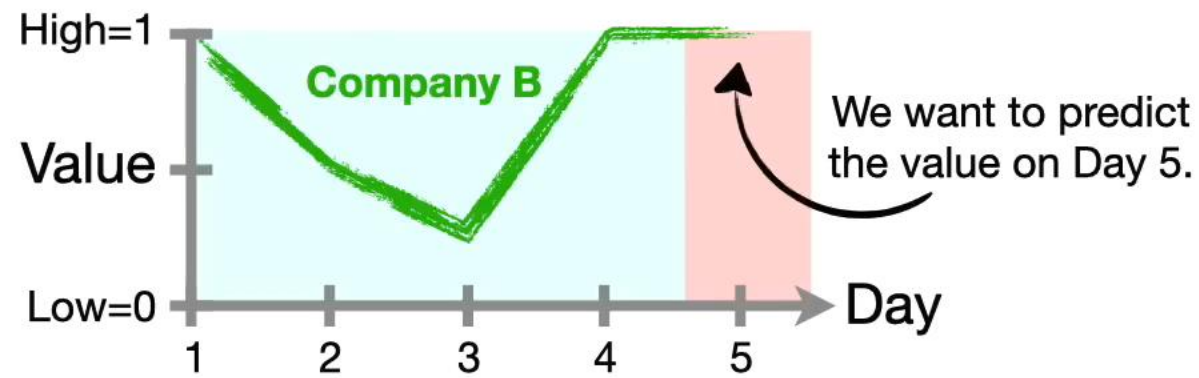
And that means that the **Output** from the **LSTM** correctly predicts **Company A's** value for **Day 5**.



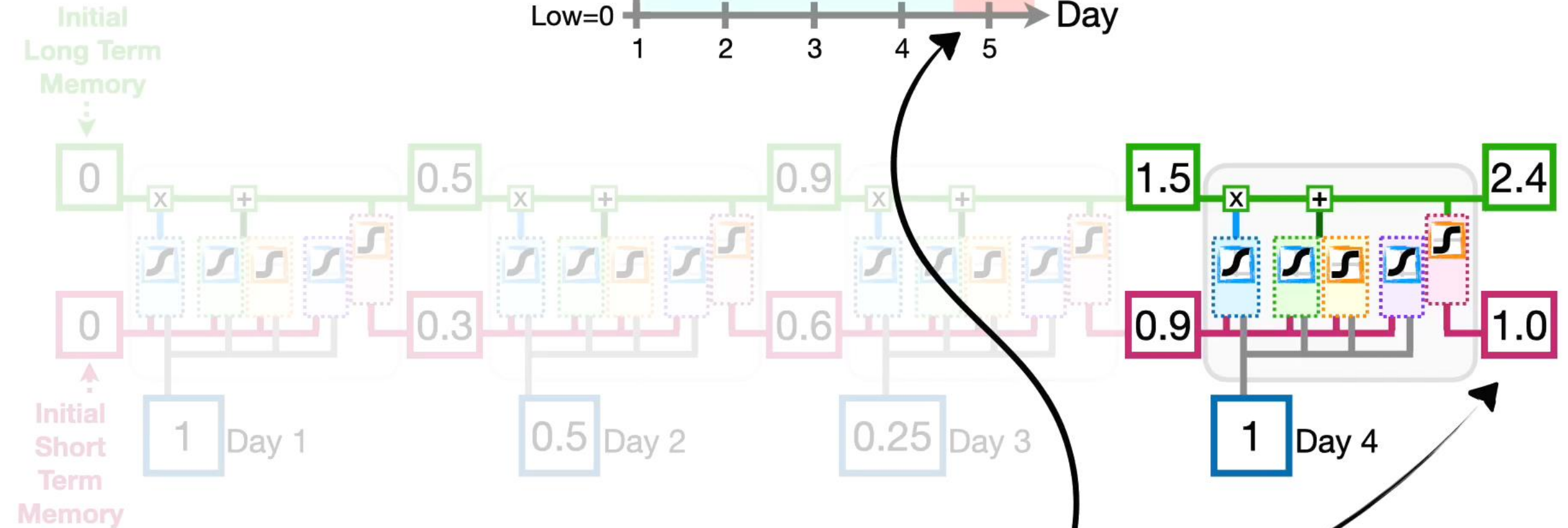
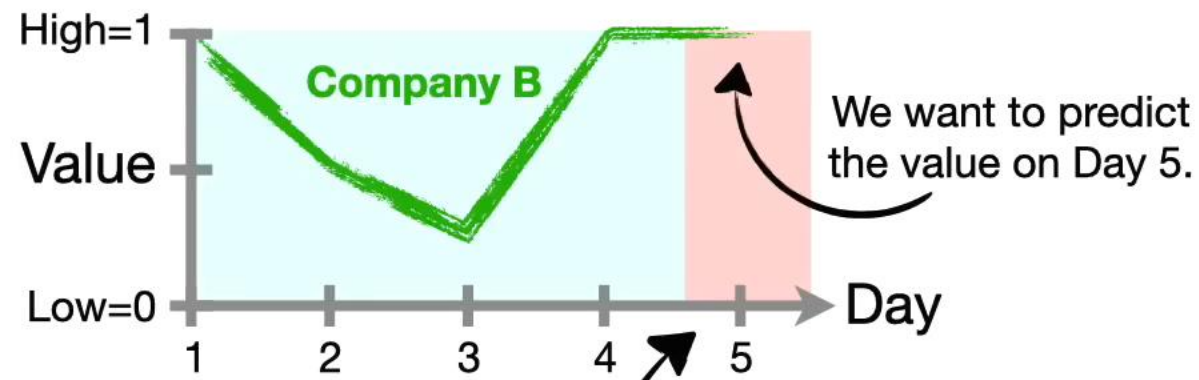
...and **0.3** (rounded) for the updated **Short-Term Memory**.



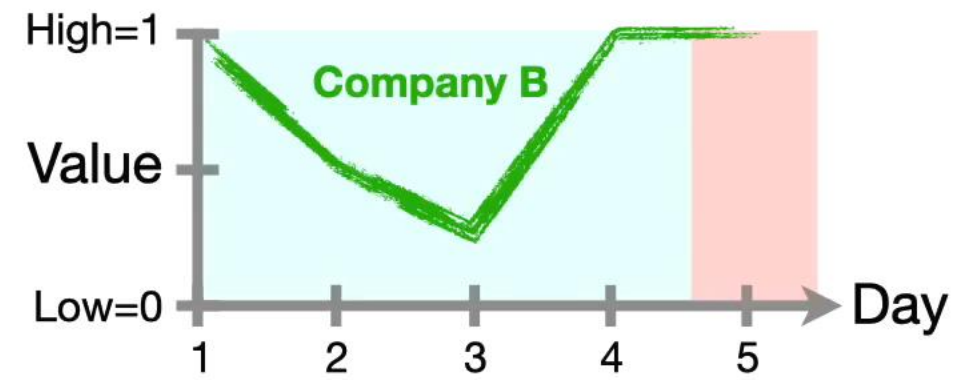
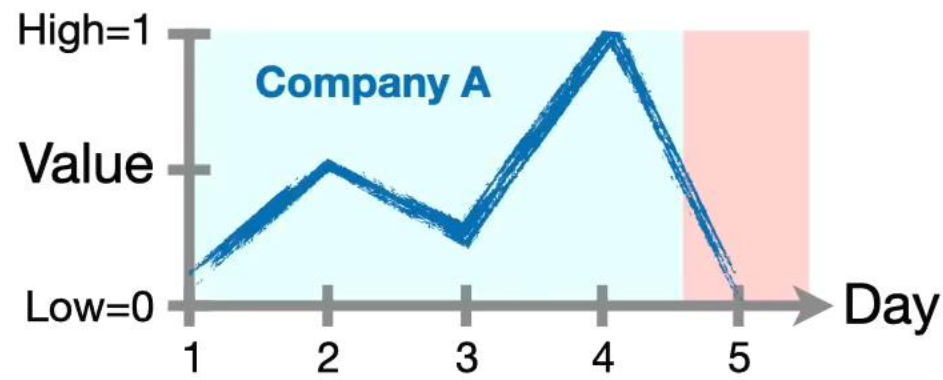
Now we unroll the **LSTM** and do the math with the remaining input values...



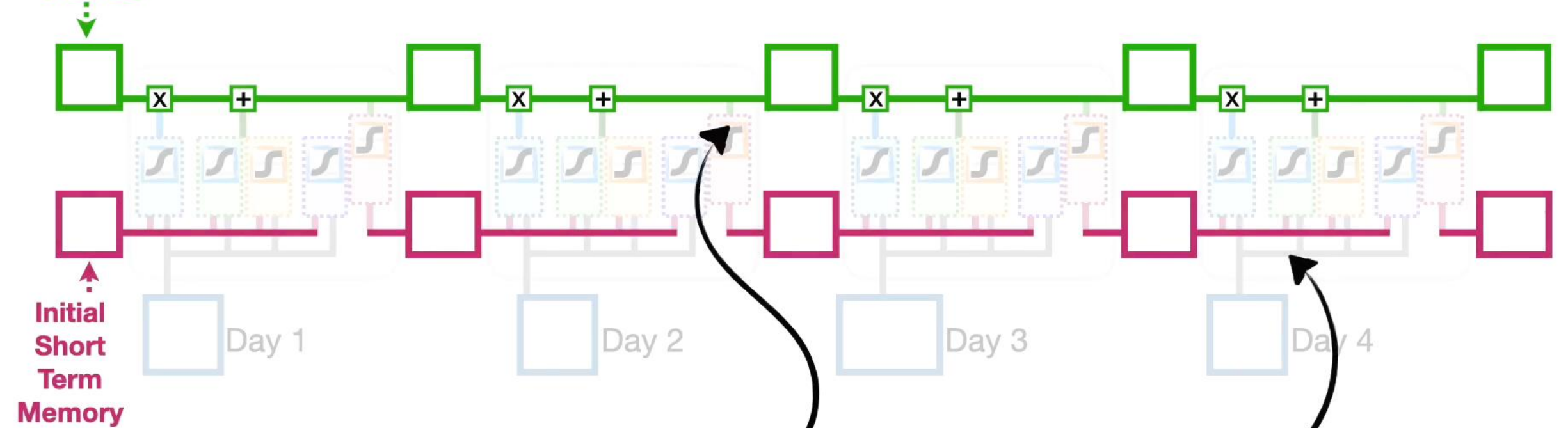
Now we unroll the **LSTM** and do the math with the remaining input values...



And that means that the **Output** from the **LSTM** correctly predicts **Company B's** value for **Day 5**.



Initial
Long Term
Memory



In summary, using
separate paths for **Long-
Term Memories...**

...and **Short-Term
Memories...**

References

- StatQuest!
 - <https://statquest.org/>
- Andrew Ng's Course on Sequence Models
 - <https://deeplearning.ai/>

Thanks for listening!